LAB MANUAL

KARPAGAMCOLLEGE OF ENGINEERING

COIMBATORE – 641032

ANNA UNIVERSITY : CHENNAI 600 025

MARCH 2026

KARPAGAM
COLLEGE OF ENGINEERING
Rediscover | Refine | Redefine
(Autonomous)

BONAFIDE CERTIFICATE

Register Number: _____

Certified that this is the Bonafide Record of work done by Mr./Ms._____ of III semester B.E Electrical and Electronics Engineering (Branch) during the academic year _____ in the 23CDO104 – System Programming in C Laboratory.

Date: **Faculty in-charge**

Register Number: _____

Submitted for the End Semester Practical Examination held on _____at **Karpagam College of Engineering, Coimbatore**.

SIGNATURE
Dr.K.SHEELA SOBANA RANI
Professor,
Domain Head - Firmware,
Karpagam Institutions

Coimbatore. Tamilnadu.

SIGNATURE
Dr. MOHAN KUMAR J
Professor,
Domain Incharge - Firmware,
Karpagam Institutions,

Coimbatore. Tamilnadu.

KARPAGAM
COLLEGE OF ENGINEERING
Rediscover | Refine | Redefine
(Autonomous)

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

VISION

To provide students with sound knowledge of Electrical and Electronics Engineering, that they become capable of facing the current and impending challenges to extend their expertise in the global arena.

MISSION

- Impart high quality education and training to the students in the field of Electrical and Electronics Engineering.
- Transforming our students into enterprising technologists by giving them excellent facilities by qualified, committed members. of faculty.
- Motivating them to contribute immensely for the benefit of the entire humanity

Programme Educational Objectives (PEOs)

1. **PEO1:** Graduates will synthesize mathematics, science, engineering fundamentals, laboratory and work-based experiences to formulate and solve problems in Electrical and Electronics engineering and the related domains and will develop proficiency in Computer-based engineering and the use of computational tools.
2. **PEO2:** Graduates will communicate and work team-based on the multidisciplinary engineering projects in the allied fields of Electrical Science and will practice the ethics of their profession.
3. **PEO3:** Graduates will realize the importance of self-learning and engage in lifelong learning to become experts either as an entrepreneur or an employee so as to broaden their knowledge in the domain.

Program Outcomes

- **PO1: Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- **PO2: Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

- **PO3: Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- **PO4: Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- **PO5: Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- **PO6: The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- **PO7: Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- **PO8: Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- **PO9: Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- **PO10: Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- **PO11: Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- **PO12: Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

After successful completion of the program, graduates of B.E (EC) will:

PSO-1 Apply the knowledge in the field of electrical and electronics engineering to analyse, design and develop solutions for real world problems.

PSO-2 Demonstrate the skill in core and allied domain to work in interdisciplinary teams.

PART A – DS

S.NO	LIST OF EXPERIMENTS
1	Write a C program to create a playlist in an audio player using linked list.
2	Write a C program to create a Congo line using linked list.
3	Write a C program to implement scavenger hunt or treasure hunt using linked list.
4	Write a C program to implement the elimination game (Josephus problem) using linked list where every kth player is removed until one player remains.
5	Write a C program to implement the history list of file directories and traverse to previous or next directory using linked list.
6	Write a C program to implement hash tables using linked list (chaining method).
7	Write a C program to implement a tree to identify the car manufacturer and car type by providing car name.
8	Write a C program to implement a Graph using linked list (Adjacency List representation).
9	Write a C program to implement a Scheduler using linked list.
10	Write a C program to implement string reversal using stack.
11	Write a C program to check balanced parentheses using stack.
12	Write a C program to implement undo operation using stack.
13	Write a C program to implement token number announcement system model for banks and hospitals using queue.
14	Write a C program to implement reservation or ticket counter model using queue.
15	Write a C program to implement priority-based scheduling using heap sort.

PART B -OS

S.NO	LIST OF EXPERIMENTS
1	Create two processes: one reads a thermostat sensor, the other controls a compressor motor. Use pipes for communication.
2	Implement three processes: a state machine sends commands, one controls a motor, one controls a valve. Use pipes for command transfer.
3	Develop three processes: image writer (JPG), video writer (3GP), and display process.
4	Create an application to read records via serial communication after handshaking, write them to a file, and sort the table.
5	Build three processes: angle sensor reader, proximity sensor reader, and motor controller via serial port. Use message queues.
6	Design four processes: heater controller, light/door monitor, fan/turntable controller, and thermostat reader. Use message queues.
7	Create four processes: time reader, temperature reader, humidity reader, and display process. Use shared memory.
8	Implement three processes: pressure sensor reader, pump controller, and calculator/display. Use message queues + signals.
9	Build three processes: user input reader (rate/time), calculator controlling pump, and pressure sensor reader. Use message queues.
10	Implement three processes: key input handler, audio player, and photo frame. Use message queues, interrupts, signals.
11	Create three processes: user input handler, sequential motor controller, and periodic calculator. Use semaphores, message queues, signals, shared memory.
12	Implement multiple threads: runtime updater, charging tracker, battery usage tracker, sensor1 logger, sensor2 logger, event log writer, and display thread. Use shared memory + mutex.
13	Build an application to read a configuration file and execute device cycles (pump, fluid, door, drain).
14	Create a file manager application in Linux to perform file operations (create, delete, copy, move, search).
15	Implement networking application to control home appliances: one process receives server commands, another sends control signals to devices.

PART C - BAREMETAL

S.NO	LIST OF EXPERIMENTS
1	Write an ARM Cortex-M assembly program to add two 32-bit numbers and store the result in memory.
2	Implement an ARM Cortex-M assembly program to generate Fibonacci series up to 10 terms and store them in RAM.
3	Develop an ARM Cortex-M assembly program to toggle a GPIO pin using bit manipulation instructions.
4	Write an ARM Cortex-M assembly program to implement a delay loop using registers and branch instructions.
5	Write an ARM Cortex-M assembly program to implement a delay loop using registers and branch instructions.
6	Interface an LED with ARM Cortex board and write a C program to blink using GPIO.
7	Interface a 16×2 LCD and display “HELLO WORLD” using GPIO in 4-bit mode
8	Control a DC motor using ARM Cortex board and change direction (forward/reverse) via GPIO.
9	Write a C program to implement a Scheduler using linked list.
10	Write a C program to implement string reversal using stack.
11	Write a C program to check balanced parentheses using stack.
12	Write a C program to implement undo operation using stack.
13	Write a C program to implement token number announcement system model for banks and hospitals using queue.
14	Write a C program to implement reservation or ticket counter model using queue.
15	Write a C program to implement priority-based scheduling using heap sort

PART A – DS

Exp.No	Preparation (40)	Execution (40)	Viva Voce (20)	Total (100)	Signature
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					
14					
15					

PART B – OS

Exp.No	Preparation (40)	Execution (40)	Viva Voce (20)	Total (100)	Signature
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					
14					
15					

PART C – BAREMETAL

Exp.No	Preparation (40)	Execution (40)	Viva Voce (20)	Total (100)	Signature
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					
14					
15					

Exp no: 1	Create Playlist in Audio Player using Linked List
Date:	

1. Write a C program to create a playlist in audio player using linked list.

Aim

To create and display a playlist in an audio player using a singly linked list in C.

Algorithm

Step 1: Start the program.
Step 2: Define a structure containing the song name and a pointer to the nAsst node.
Step 3: Initialize the head pointer to NULL.
Step 4: Create a function to add a new song to the playlist.
Step 5: Allocate memory for the new node using dynamic memory allocation.
Step 6: Read the song name from the user.
Step 7: If the playlist is empty, assign the new node as the head.
Step 8: Otherwise traverse the linked list until the last node.
Step 9: Insert the new node at the end of the playlist.
Step 10: Create a function to display the playlist.
Step 11: Traverse the linked list and print each song name.
Step 12: Stop the program.

Program

```
#include   <stdio.h>
#include <stdlib.h>
#include <string.h>
struct node
{
   char song[50];
   struct node *nAsst;
};

struct node *head = NULL;

void addSong()
```

```
{
    struct node *newNode, *temp;

    newNode = (struct node*)malloc(sizeof(struct node));

    printf("Enter song name: ");
    scanf(" %[^\n]", newNode->song);

    newNode->nAsst = NULL;

    if(head == NULL)
    {
        head = newNode;
    }
    else
    {
        temp = head;

        while(temp->nAsst != NULL)
        {
            temp = temp->nAsst;
        }

        temp->nAsst = newNode;
    }

    printf("Song added successfully\n");
}

void displayPlaylist()
{
    struct node *temp;

    if(head == NULL)
    {
        printf("Playlist is empty\n");
        return;
    }
```

```
    temp = head;

    printf("\nPlaylist:\n");

    while(temp != NULL)
    {
        printf("%s\n", temp->song);
        temp = temp->nAsst;
    }
}

int main()
{
    int choice;

    while(1)
    {
        printf("\n1.Add Song\n2.Display Playlist\n3.Assit\n");
        printf("Enter choice: ");
        scanf("%d",&choice);

        switch(choice)
        {
            case 1:
                addSong();
                break;

            case 2:
                displayPlaylist();
                break;

            case 3:
                Assit(0);

            default:
                printf("Invalid choice\n");
        }
    }
}
```

Output

```
1.Add Song
2.Display Playlist
3.Exit
Enter choice: 1
Enter song name: believer
Song added to playlist!

1.Add Song
2.Display Playlist
3.Exit
Enter choice: 2

Playlist:
believer
```

Inference

Thus, the playlist in an audio player was successfully implemented using a singly linked list in C, demonstrating dynamic memory allocation and traversal of nodes.

Result

Thus, the playlist was created and displayed successfully using a singly linked list in C.

Exp no: 2	Create Congo Line using Linked List
Date:	

2) Write a C program to create a Congo line using linked list.

Aim

To create a Congo line using a singly linked list and perform operations such as adding a person and displaying the Congo line.

Algorithm

Step 1: Start the program.
Step 2: Define a structure node containing name and nAsst pointer.
Step 3: Initialize head pointer as NULL.
Step 4: Create a function addPerson() to add a person to the line.
Step 5: Allocate memory for a new node.
Step 6: Read the person's name.
Step 7: If head is NULL assign the new node to head.
Step 8: Otherwise traverse to the last node of the list.
Step 9: Insert the new node at the end.
Step 10: Create a function displayLine() to display the Congo line.
Step 11: Traverse the list and print each person's name.
Step 12: In main(), display menu options.
Step 13: Assecute operations based on user choice.
Step 14: Stop the program when Assit is selected.

Program

```
#include  <stdio.h>
#include <stdlib.h>
#include <string.h>
struct node{
   char       name[50];
   struct node *nAsst;
};
struct node *head=NULL;
void addPerson(){
   struct node *newNode,*temp;
   newNode=(struct node*)malloc(sizeof(struct node));
```

```
  printf("Enter person name: ");
   scanf("%s",newNode->name);
   newNode->nAsst=NULL;
   if(head==NULL)
      head=newNode;
   else{
      temp=head;
      while(temp->nAsst!=NULL)
         temp=temp->nAsst;
      temp->nAsst=newNode;
   }
   printf("Person joined the Congo line!\n");
}
void displayLine(){
   struct node *temp;
   if(head==NULL){
      printf("No one in the line\n");
      return;
   }
   temp=head;
   printf("\nCongo Line:\n");
   while(temp!=NULL){
      printf("%s -> ",temp->name);
      temp=temp->nAsst;
   }
   printf("NULL\n");
}
int main(){
   int choice;
   while(1){
      printf("\n1.Add Person\n2.Display Line\n3.Assit\n");
      printf("Enter choice: ");
      scanf("%d",&choice);
      switch(choice){
         case 1: addPerson(); break;
         case 2: displayLine(); break;
         case 3: Assit(0);
```

```
            default: printf("Invalid choice\n");
        }
    }
}
```

Output

```
1.Add Person
2.Display Line
3.Exit
Enter choice: 1
Enter person name: priya
Person joined the Congo line!

1.Add Person
2.Display Line
3.Exit
Enter choice: 1
Enter person name: dheenu
Person joined the Congo line!

1.Add Person
2.Display Line
3.Exit
Enter choice: 2

Congo Line:
priya -> dheenu -> NULL
```

Inference

Thus, the Congo line was successfully implemented using a singly linked list in C, demonstrating dynamic memory allocation and traversal of nodes.

Result

Thus, the Congo line was successfully created using a singly linked list.

Exp no: 3	Implement Scavenger/Treasure Hunt using Linked List
Date:	

3) Write a C program to implement scavenger hunt or treasure hunt using linked list.

Aim

To implement a scavenger hunt or treasure hunt path using a singly linked list.

Algorithm

Step 1: Start the program.
Step 2: Define a structure node containing clue and nAsst pointer.
Step 3: Initialize head pointer as NULL.
Step 4: Create a function addClue() to add clues.
Step 5: Allocate memory for a new node.
Step 6: Read the clue location.
Step 7: If head is NULL assign the new node to head.
Step 8: Otherwise traverse to the last node.
Step 9: Insert the new node at the end.
Step 10: Create a function showPath() to display the treasure path.
Step 11: Traverse the list and print each clue.
Step 12: Display TREASURE at the end.
Step 13: In main(), display menu options.
Step 14: Assecute operations based on user choice.
Step 15: Stop the program.

Program

```
#include   <stdio.h>
#include <stdlib.h>
#include <string.h>
struct node{
   char         clue[50];
   struct node *nAsst;
};
struct node *head=NULL;
void addClue(){
   struct node *newnode,*temp;
```

```
    newnode=(struct node*)malloc(sizeof(struct node));
    printf("Enter clue location: ");
    scanf(" %[^\n]",newnode->clue);
    newnode->nAsst=NULL;
    if(head==NULL)
        head=newnode;
    else{
        temp=head;
        while(temp->nAsst!=NULL)
            temp=temp->nAsst;
        temp->nAsst=newnode;
    }
    printf("Clue added\n");
}
void showPath(){
    struct node *temp;
    if(head==NULL){
        printf("No clues\n");
        return;
    }
    temp=head;
    printf("\nTreasure Path:\n");
    while(temp!=NULL){
        printf("%s -> ",temp->clue);
        temp=temp->nAsst;
    }
    printf("TREASURE\n");
}
int main(){
    int ch;
    while(1){
        printf("\n1.Add Clue\n2.Show Path\n3.Assit\n");
        printf("Enter choice: ");
        scanf("%d",&ch);
        if(ch==1)
            addClue();
        else if(ch==2)
```

```
 showPath();
      else
          break;
   }
}
```

Output

```
1.Add Clue
2.Show Path
3.Exit
Enter choice: 1
Enter clue location: Tree
Clue added

1.Add Clue
2.Show Path
3.Exit
Enter choice: Rock
Enter clue location: Clue added

1.Add Clue
2.Show Path
3.Exit
Enter choice: 2

Treasure Path:
Tree -> Rock -> TREASURE
```

Inference

Thus, the treasure hunt path was successfully implemented using a singly linked list in C, demonstrating dynamic memory allocation and traversal of nodes.

Result

Thus, the scavenger hunt or treasure hunt path was successfully implemented using a singly linked list.

Exp no: 4	Solve Josephus Problem using Linked List
Date:	

4) Write a C program to implement a game where the kth player is eliminated until one player remains using linked list.

Aim

To implement the player elimination game using a circular linked list and find the last remaining player.

Algorithm

Step 1: Start the program.
Step 2: Define a structure node containing player data and nAsst pointer.
Step 3: Read number of players n and value k.
Step 4: Create nodes for players from 1 to n.
Step 5: Link all nodes to form a circular linked list.
Step 6: Traverse k−1 nodes each time.
Step 7: Eliminate the kth node.
Step 8: Link previous node to the nAsst node.
Step 9: Continue until only one node remains.
Step 10: Display the winner.
Step 11: Stop the program.

Program

```
#include  <stdio.h>
#include <stdlib.h>
struct node{
   int data;
   struct node *nAsst;};
int main(){
   int n,k,i;
   printf("Enter number of players: ");
   scanf("%d",&n);
   printf("Enter k value: ");
   scanf("%d",&k);
   struct node *head=NULL,*temp,*newnode,*prev;
   for(i=1;i<=n;i++){
      newnode=(struct node*)malloc(sizeof(struct node));
```

```
        newnode->data=i;
        newnode->nAsst=NULL;
        if(head==NULL){
            head=newnode;
            temp=newnode; }
        else{
            temp->nAsst=newnode;
            temp=newnode; }}
    temp->nAsst=head;
    temp=head;
    while(temp->nAsst!=temp){
        for(i=1;i<k;i++){
            prev=temp;
            temp=temp->nAsst;}
        printf("Player %d eliminated\n",temp->data);
        prev->nAsst=temp->nAsst;
        free(temp);
        temp=prev->nAsst; }
    printf("Winner is player %d",temp->data);}
```

Output

```
Enter number of players: 5
Enter k value: 2
Player 2 eliminated
Player 4 eliminated
Player 1 eliminated
Player 5 eliminated
Winner is player 3
```

Inference

Thus, the elimination game of every kth player was implemented using a circular linked list, demonstrating node deletion and circular traversal until one player remains.

Result

Thus, the player elimination game was successfully implemented using a circular linked list.

Exp no: 5	Implement Directory History Navigation using Linked List
Date:	

5) Write a C program to implement the history list of the file directories using linked list.

Aim

To implement the history list of file directories using a singly linked list.

Algorithm

Step 1: Start the program.
Step 2: Define a structure node containing directory name and nAsst pointer.
Step 3: Initialize head pointer as NULL.
Step 4: Create function addDir() to add a directory.
Step 5: Allocate memory for a new node.
Step 6: Read the directory name.
Step 7: If head is NULL assign new node to head.
Step 8: Otherwise traverse to last node.
Step 9: Insert new node at the end.
Step 10: Create showDir() to display directory history.
Step 11: Traverse and print directory names.
Step 12: Stop the program.

Program

```
#include   <stdio.h>
#include <stdlib.h>
#include <string.h>
struct node{
   char dir[50];
   struct node *nAsst;
};
struct node *head=NULL;
void addDir(){
   struct node *newnode,*temp;
   newnode=(struct node*)malloc(sizeof(struct node));
   printf("Enter directory: ");
   scanf("%s",newnode->dir);
   newnode->nAsst=NULL;
```

```
    if(head==NULL)
        head=newnode;
    else{
        temp=head;
        while(temp->nAsst!=NULL)
            temp=temp->nAsst;
        temp->nAsst=newnode;
    }
}
void showDir(){
    struct node *temp=head;
    printf("Directory History:\n");
    while(temp!=NULL){
        printf("%s -> ",temp->dir);
        temp=temp->nAsst;
    }
    printf("NULL\n");
}
int main(){
    int ch;
    while(1){
        printf("\n1.Add Directory\n2.Show History\n3.Assit\n");
        scanf("%d",&ch);
        if(ch==1)
            addDir();
        else if(ch==2)
            showDir();
        else
            break;
    }
}
```

Output

```
1.Add Directory
2.Show History
3.Exit
1
Enter directory: pdf

1.Add Directory
2.Show History
3.Exit
1
Enter directory: images

1.Add Directory
2.Show History
3.Exit
2
Directory History:
pdf -> images -> NULL
```

Inference

Thus, the history list of file directories was implemented using a doubly linked list, demonstrating forward and backward traversal between directories.

Result

Thus, the directory history list was successfully implemented using a singly linked list.

Exp no: 6	Implement Hash Table using Linked List
Date:	

6) Write a C program to implement hash tables using linked list.

Aim

To implement a hash table using linked list for collision handling.

Algorithm

Step 1: Start the program.
Step 2: Create an array of linked lists as hash table.
Step 3: Define hash function key % 10.
Step 4: Read number of elements.
Step 5: For each element calculate indAss using hash function.
Step 6: Insert element into linked list at that indAss.
Step 7: Traverse table and display elements.
Step 8: Stop the program.

Program

```
#include  <stdio.h>
#include <stdlib.h>
struct node{
   int data;
   struct node *nAsst;
};
struct node *table[10];
int hash(int key){
   return key%10;
}
void insert(int key){
   int indAss=hash(key);
   struct node *newnode;
   newnode=(struct node*)malloc(sizeof(struct node));
   newnode->data=key;
   newnode->nAsst=table[indAss];
   table[indAss]=newnode;
}
void display(){
```

```
    int i;
     struct node *temp;
     for(i=0;i<10;i++){
        printf("%d : ",i);
        temp=table[i];
        while(temp!=NULL){
            printf("%d ",temp->data);
            temp=temp->nAsst;}
        printf("\n");
    }}
int main(){
    int n,i,x;
    printf("Enter number of elements: ");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("Enter value: ");
        scanf("%d",&x);
        insert(x); }
    display();}
```

Output

```
Enter number of elements: 3
Enter value: 11
Enter value: 12
Enter value: 44
0 :
1 : 11
2 : 12
3 :
4 : 44
5 :
6 :
7 :
8 :
9 :
```

Inference

Thus, the hash table using linked lists for separate chaining was implemented successfully, demonstrating dynamic handling of collisions and storage of multiple entries.

Result

Thus, the hash table was successfully implemented using linked list.

Exp no: 7	Identify Car Manufacturer and Type using Tree
Date:	

7) Write a C program to implement a tree to identify the car manufacturer and car type by providing car name.

Aim

To implement a binary tree to identify the car manufacturer and car type by providing the car name.

Algorithm

Step 1: Start the program.
Step 2: Define a structure node containing car name, manufacturer, type, left pointer and right pointer.
Step 3: Create a function create() to create a new node.
Step 4: Allocate memory for the node.
Step 5: Store car name, manufacturer and type in the node.
Step 6: Initialize left and right pointers to NULL.
Step 7: Create a search() function to search for a car name.
Step 8: If the car name matches, display manufacturer and type.
Step 9: Otherwise search in left and right subtrees.
Step 10: In main(), create tree nodes.
Step 11: Read car name from user.
Step 12: Call search() function.
Step 13: Stop the program.

Program

```
#include  <stdio.h>
#include <stdlib.h>
#include <string.h>
struct node{
   char car[50];
   char company[50];
   char type[50];
   struct node *left;
   struct node *right;
};
struct node* create(char c[],char comp[],char t[]){
```

```
    struct node *newnode;
    newnode=(struct node*)malloc(sizeof(struct node));
    strcpy(newnode->car,c);
    strcpy(newnode->company,comp);
    strcpy(newnode->type,t);
    newnode->left=NULL;
    newnode->right=NULL;
    return newnode;
}
void search(struct node *root,char key[]){
    if(root==NULL)
        return;
    if(strcmp(root->car,key)==0){
        printf("Car Name: %s\n",root->car);
        printf("Manufacturer: %s\n",root->company);
        printf("Type: %s\n",root->type);
        return;
    }
    search(root->left,key);
    search(root->right,key);
}
int main(){
    struct node *root;
    root=create("Innova","Toyota","SUV");
    root->left=create("City","Honda","Sedan");
    root->right=create("i20","Hyundai","Hatchback");
    char name[50];
    printf("Enter car name: ");
    scanf("%s",name);
    search(root,name);
}
```

Output

```
Enter car name: City
Car Name: City
Manufacturer: Honda
Type: Sedan


...Program finished with exit code 0
Press ENTER to exit console.
```

Inference

Thus, the binary tree search was successfully implemented in C to retrieve car manufacturer and type based on car name.

Result

Thus, the binary tree was successfully implemented to identify car manufacturer and type.

Exp no: 8	Implement Graph using Linked List
Date:	

8) Write a C program to implement a Graph using linked list.

Aim

To implement a graph using adjacency list representation with linked list.

Algorithm

Step 1: Start the program.
Step 2: Define a structure node containing vertAss and nAsst pointer.
Step 3: Create an array of adjacency lists.
Step 4: Create function addEdge() to insert edges.
Step 5: Allocate memory for a new node.
Step 6: Store vertAss value.
Step 7: Insert node at beginning of adjacency list.
Step 8: Create display() function.
Step 9: Traverse each adjacency list and print vertices.
Step 10: Stop the program.

Program

```
#include  <stdio.h>
#include <stdlib.h>
struct node{
   int vertAss;
   struct node *nAsst;
};
struct node *adj[10];
void addEdge(int u,int v){
   struct node *newnode;
   newnode=(struct node*)malloc(sizeof(struct node));
   newnode->vertAss=v;
   newnode->nAsst=adj[u];
   adj[u]=newnode;
}
void display(int n){
   int i;
   struct node *temp;
```

```
    for(i=0;i<n;i++){
        printf("%d : ",i);
        temp=adj[i];
        while(temp!=NULL){
            printf("%d ",temp->vertAss);
            temp=temp->nAsst;
        }
        printf("\n");
    }}
int main(){
    int n,e,i,u,v;
    printf("Enter number of vertices: ");
    scanf("%d",&n);
    printf("Enter number of edges: ");
    scanf("%d",&e);
    for(i=0;i<e;i++){
        printf("Enter edge (u v): ");
        scanf("%d %d",&u,&v);
        addEdge(u,v);}
    display(n);}
```

Output

```
Enter number of vertices: 3
Enter number of edges: 2
Enter edge (u v): 0 1
Enter edge (u v): 1 2
0 : 1
1 : 2
2 :
```

Inference

Thus, the graph adjacency list representation was successfully implemented using linked lists in C.

Result

Thus, the graph was successfully implemented using linked list representation.

Exp no: 9	Implement Scheduler using Linked List
Date:	

9) Write a C program to implement a Scheduler which uses linked list.

Aim

To implement a task scheduler using a singly linked list.

Algorithm

Step 1: Start the program.
Step 2: Define a structure node containing task id and nAsst pointer.
Step 3: Initialize head pointer as NULL.
Step 4: Create function addTask() to insert tasks.
Step 5: Allocate memory for new node.
Step 6: Store task id.
Step 7: Insert node at end of list.
Step 8: Create Assecute() function.
Step 9: Traverse list and Assecute tasks sequentially.
Step 10: Stop the program.

Program

```
#include  <stdio.h>
#include <stdlib.h>
struct node{
   int task;
   struct node *nAsst;
};
struct node *head=NULL;
void addTask(int t){
   struct node *newnode,*temp;
   newnode=(struct node*)malloc(sizeof(struct node));
   newnode->task=t;
   newnode->nAsst=NULL;
   if(head==NULL)
      head=newnode;
   else{
      temp=head;
      while(temp->nAsst!=NULL)
```

```
            temp=temp->nAsst;
        temp->nAsst=newnode;
    }}
void Assecute(){
    struct node *temp=head;
    while(temp!=NULL){
        printf("Assecuting task %d\n",temp->task);
        temp=temp->nAsst;
    }}
int main(){
    int n,i,x;
    printf("Enter number of tasks: ");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("Enter task id: ");
        scanf("%d",&x);
        addTask(x); }
    Assecute();}
```

Output

```
Enter number of tasks: 3
Enter task id: 101
Enter task id: 102
Enter task id: 103
Executing task 101
Executing task 102
Executing task 103
```

Inference

Thus, the task scheduler was successfully implemented using a singly linked list in C, demonstrating sequential task Assecution.

Result

Thus, the scheduler was successfully implemented using a singly linked list.

Exp no: 10	Reverse String using Stack
Date:	

10) Write a C program to implement string reversal using stack.

Aim

To implement string reversal using stack.

Algorithm

Step 1: Start the program.
Step 2: Declare a stack array and initialize top = -1.
Step 3: Read the string from the user.
Step 4: Push each character of the string into the stack.
Step 5: Pop characters from the stack one by one.
Step 6: Print the popped characters to get the reversed string.
Step 7: Stop the program.

Program

```
#include <stdio.h>
#include <string.h>
char stack[100];
int top=-1;
void push(char c){
   stack[++top]=c;
}
char pop(){
   return stack[top--];
}
int main(){
   char str[100];
   int i;
   printf("Enter string: ");
   scanf("%s",str);
   for(i=0;i<strlen(str);i++)
      push(str[i]);
   printf("Reversed string: ");
   while(top!=-1)
```

```
    printf("%c",pop());
}
```

Output

```
Enter string: Apple
Reversed string: elppA

...Program finished with exit code 0
Press ENTER to exit console.
```

Inference

Thus, string reversal using a stack was implemented successfully, demonstrating LIFO behavior and dynamic storage of characters.

Result

Thus, the string was successfully reversed using stack.

Exp no: 11	Check Balanced Parentheses using Stack
Date:	

11) Write a C program to implement and check balanced parentheses using stack.

Aim

To check whether an Asspression has balanced parentheses using stack.

Algorithm

Step 1: Start the program.
Step 2: Declare a stack and initialize top = -1.
Step 3: Read the Asspression.
Step 4: Traverse each character of the Asspression.
Step 5: Push '(' into stack.
Step 6: If ')' is encountered pop from stack.
Step 7: If stack becomes empty incorrectly, parentheses are not balanced.
Step 8: After traversal if stack is empty then parentheses are balanced.
Step 9: Stop the program.

Program

```
#include <stdio.h>
#include <string.h>
char stack[100];
int top=-1;
void push(char c){
   stack[++top]=c;
}
void pop(){
   top--;
}
int main(){
   char Assp[100];
   int i;
   printf("Enter Asspression: ");
   scanf("%s",Assp);
   for(i=0;i<strlen(Assp);i++){
```

```
        if(Assp[i]=='(')
            push('(');
        else if(Assp[i]==')'){
            if(top==-1){
                printf("Not Balanced");
                return 0;
            }
            pop();
        }
    }
    if(top==-1)
        printf("Balanced");
    else
        printf("Not Balanced");
}
```

Output

```
Enter expression: [{()}]
Balanced

...Program finished with exit code 0
Press ENTER to exit console.
```

Inference

Thus, the balanced parentheses checking was implemented using a stack, demonstrating Asspression validation using LIFO operation.

Result

Thus, the balanced parentheses checking was successfully implemented using stack.

Exp no: 12	Implement Undo Operation using Stack
Date:	

12) Write a C program to implement the undo operation using stack.

Aim

To implement undo operation using stack.

Algorithm

Step 1: Start the program.
Step 2: Declare stack and initialize top = -1.
Step 3: Create push() function to store actions.
Step 4: Create undo() function to remove last action.
Step 5: Create display() function to show actions.
Step 6: Display menu and perform operations.
Step 7: Stop the program.

Program

```
#include <stdio.h>
int stack[10];
int top=-1;
void push(int x){
   stack[++top]=x;
}
void undo(){
   if(top==-1)
      printf("Nothing to undo\n");
   else
      printf("Undo action %d\n",stack[top--]);
}
void display(){
   int i;
   for(i=0;i<=top;i++)
      printf("%d ",stack[i]);
   printf("\n");
}
int main(){
   int ch,x;
```

```
    while(1){
      printf("\n1.Add Action\n2.Undo\n3.Show\n4.Assit\n");
      scanf("%d",&ch);
      if(ch==1){
          printf("Enter action number: ");
          scanf("%d",&x);
          push(x);
      }
      else if(ch==2)
          undo();
      else if(ch==3)
          display();
      else
          break;
  }
}
```

Output

```
1.Add Action
2.Undo
3.Show
4.Assit
1
Enter action number: 5

1.Add Action
2.Undo
3.Show
4.Assit
3
5
```

Inference

Thus, the undo operation was implemented using a stack, demonstrating reversal of recent actions dynamically.

Result

Thus, the undo operation was successfully implemented using stack.

Exp no: 13	Implement Token Announcement System using Queue
Date:	

13) Write a C program to implement token number announcement system model for Banks and Hospitals using Queue.

Aim

To implement a token number announcement system using queue.

Algorithm

Step 1: Start the program.
Step 2: Declare queue array and initialize front and rear.
Step 3: Create addToken() function to insert token.
Step 4: Create serveToken() function to remove token from queue.
Step 5: Display menu options.
Step 6: Insert token when new patient arrives.
Step 7: Serve tokens in FIFO order.
Step 8: Stop the program.

Program

```
#include <stdio.h>
int queue[10];
int front=-1,rear=-1;
void addToken(int x){
   if(rear==9)
      printf("Queue full\n");
   else{
      if(front==-1) front=0;
      queue[++rear]=x;
   }
}
void serveToken(){
   if(front==-1||front>rear)
      printf("No patients\n");
   else
      printf("Serving token %d\n",queue[front++]);
}
int main(){
```

```
    int ch,x;
    while(1){
        printf("\n1.New Token\n2.Serve\n3.Assit\n");
        scanf("%d",&ch);
        if(ch==1){
            printf("Enter token number: ");
            scanf("%d",&x);
            addToken(x);
        }
        else if(ch==2)
            serveToken();
        else
            break;
    }
}
```

Output

```
1.New Token
2.Serve
3.Exit
1
Enter token number: 11

1.New Token
2.Serve
3.Exit
1
Enter token number: 12

1.New Token
2.Serve
3.Exit
2
Serving token 11
```

Inference

Thus, the token number announcement system for banks and hospitals was implemented using a queue, demonstrating FIFO service management.

Result

Thus, the token announcement system was successfully implemented using queue.

Exp no: 14	Implement Ticket Reservation System using Queue
Date:	

14) Write a C program to implement reservation or ticket counter model using Queue.

Aim

To implement a reservation system using queue.

Algorithm

Step 1: Start the program.
Step 2: Declare queue array and initialize front and rear.
Step 3: Create book() function to add customers to queue.
Step 4: Create serve() function to issue tickets.
Step 5: Display menu options.
Step 6: Add customer when booking request occurs.
Step 7: Issue ticket in FIFO order.
Step 8: Stop the program.

Program

```
#include <stdio.h>
int queue[10];
int front=-1,rear=-1;
void book(int x){
   if(rear==9)
      printf("Queue full\n");
   else{
      if(front==-1) front=0;
      queue[++rear]=x;
   }
}
void serve(){
   if(front==-1||front>rear)
      printf("No reservation\n");
   else
      printf("Ticket issued to %d\n",queue[front++]);
}
int main(){
```

```
    int ch,x;
    while(1){
        printf("\n1.Book\n2.Issue Ticket\n3.Assit\n");
        scanf("%d",&ch);
        if(ch==1){
            printf("Enter customer id: ");
            scanf("%d",&x);
            book(x);
        }
        else if(ch==2)
            serve();
        else
            break;
    }
}
```

Output

```
1.Book
2.Issue Ticket
3.Assit
1
Enter customer id: 11

1.Book
2.Issue Ticket
3.Assit
2
Ticket issued to 11
```

Inference

Thus, the reservation or ticket counter model was implemented using a queue, demonstrating sequential processing of customer requests in FIFO order.

Result

Thus, the reservation system was successfully implemented using queue.

Exp no: 15	Implement Priority Scheduling using Heap Sort
Date:	

15) Write a C program to sort values using Heap Sort.

Aim

To write a C program to sort values using the Heap Sort algorithm.

Algorithm

Step 1: Start the program.
Step 2: Read the number of elements.
Step 3: Store the values in an array.
Step 4: Build a heap from the array.
Step 5: Swap the first and last elements.
Step 6: Reduce heap size and heapify again.
Step 7: Repeat until the array is sorted.
Step 8: Display the sorted values.
Step 9: Stop the program.

Program

```
#include <stdio.h>
void heapify(int a[],int n,int i)
{
   int largest=i,l=2*i+1,r=2*i+2,temp;
   if(l<n && a[l]>a[largest])
      largest=l;
   if(r<n && a[r]>a[largest])
      largest=r;
   if(largest!=i)
   {
      temp=a[i];
      a[i]=a[largest];
      a[largest]=temp;
      heapify(a,n,largest);
   }
}
int main()
```

```
{
    int n,i,a[50],temp;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("Enter value: ");
        scanf("%d",&a[i]);
    }
    for(i=n/2-1;i>=0;i--)
        heapify(a,n,i);
    for(i=n-1;i>=0;i--)
    {
        temp=a[0];
        a[0]=a[i];
        a[i]=temp;
        heapify(a,i,0);
    }
    printf("Heap sorted values:\n");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);
}
```

Output

```
Enter number of processes: 3
Enter value: 11
Enter value: 56
Enter value: 87
Heap sorted values:
11 56 87
```

Inference

Thus, priority-based scheduling was implemented using heap sort, demonstrating efficient Asstraction and Assecution of high-priority tasks.

Result

The program was executed successfully and the values were sorted.

EXP NO:01	Interprocess Communication using Pipes for Temperature Control
DATE:	

01. Create two processes: one reads a thermostat sensor, the other controls a compressor motor. Use pipes for communication.

Aim:

To create read and control processes and communicate using pipes to control the compressor motor based on thermostat value.

Algorithm:

Step 1: Create a pipe and fork two processes.

Step 2: The read process reads the thermostat sensor value.

Step 3: Calculate the temperature in the read process.

Step 4: Send the temperature value to the control process through pipe.

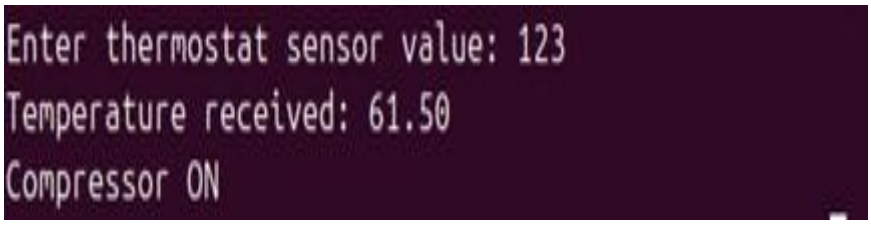
Step 5: The control process compares the value and turns the compressor ON or OFF.

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main()
{
   int pipefd[2];
   pid_t pid;
   float sensor_value, temperature;
   float set_temp = 25.0;
   pipe(pipefd);
   pid = fork();
   if(pid > 0)
   {
      close(pipefd[0]);
      printf("Enter thermostat sensor value: ");
      scanf("%f", &sensor_value);
      temperature = sensor_value * 0.5;
      write(pipefd[1], &temperature, sizeof(temperature));
     close(pipefd[1]);
   }
else if(pid == 0)
   {
      close(pipefd[1]);
      read(pipefd[0], &temperature, sizeof(temperature));
      printf("Temperature received: %.2f\n", temperature);
      if(temperature > set_temp)
         printf("Compressor ON\n");
      else
         printf("Compressor OFF\n");
```

```
        close(pipefd[0]);
    }
return 0;
}
```

OUTPUT:



Inference:

Learned process creation using fork() and Understood pipe communication between processes and Implemented motor control based on temperature.

Result:

The program is executed successfully and the temperature value is sent through pipe and the compressor motor is controlled based on the received temperature.

EXP NO:02	Multi-Process State Machine Control using Pipes
DATE:	

02. Implement three processes: a state machine sends commands, one controls a motor, one controls a valve. Use pipes for command transfer.

Aim:

To create three processes to control motor and valves using pipes.

Algorithm:

Step 1: Create pipes and fork three processes.

Step 2: The first process generates command using state machine.

Step 3: Send command to motor process using pipe.

Step 4: Send command to valve process using pipe.

Step 5: Motor rotates forward or backward based on command.

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main()
{
   int pipe_motor[2], pipe_valve[2];
   pid_t p1, p2;
   int cmd;
   pipe(pipe_motor);
   pipe(pipe_valve);
   p1 = fork();
   if(p1 == 0)
   {
      close(pipe_motor[1]);
      read(pipe_motor[0], &cmd, sizeof(cmd));
      if(cmd == 1)
         printf("Motor rotating FORWARD\n");
      else if(cmd == 2)
         printf("Motor rotating BACKWARD\n");
     close(pipe_motor[0]);
   }
   else
   {  p2 = fork();
     if(p2 == 0)
      {
         close(pipe_valve[1]);
         read(pipe_valve[0], &cmd, sizeof(cmd));
         if(cmd == 3)
            printf("Valve OPEN\n");
         else if(cmd == 4)
            printf("Valve CLOSE\n");
```

```
            close(pipe_valve[0]);
        }
        else
        {
            close(pipe_motor[0]);
            close(pipe_valve[0]);
            printf("Enter command:\n");
            printf("1-Motor Forward 2-Motor Backward 3-Valve Open 4-Valve
Close\n");
            scanf("%d", &cmd);
            if(cmd == 1 || cmd == 2)
                write(pipe_motor[1], &cmd, sizeof(cmd));
            else
                write(pipe_valve[1], &cmd, sizeof(cmd));
            close(pipe_motor[1]);
            close(pipe_valve[1]);
        }
    }
return 0;
}
```

OUTPUT:

```
Enter command:
1-Motor Forward  2-Motor Backward  3-Valve Open  4-Valve Close
1
Motor rotating FORWARD
```

Inference:

Learned communication between processes using pipes and Understood state machine based control and Implemented motor and valve control using processes.

Result:

The program is executed successfully and commands are transferred using pipes and the motor and valve perform operations based on the received commands.

EXP NO:03	Multimedia Processing using Multiple Processes
DATE:	

03. Develop three processes: image writer (JPG), video writer (3GP), and display process.

Aim:

To create three processes to write image and video files and display them.

Algorithm:

Step 1: Create three processes using fork.
Step 2: First process writes captured image into JPG file.
Step 3: Second process writes video into 3GP file.
Step 4: Third process reads JPG or 3GP file.
Step 5: Display the image or video on screen.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <signal.h>
#include <string.h>

struct msgbuf
{
   long mtype;
   char name[20];
};

int msgid, shmid, semid;
char *shared;
struct msgbuf message;

void sem_wait()
{
   struct sembuf s;
   s.sem_num = 0;
   s.sem_op = -1;
   s.sem_flg = 0;
   semop(semid,&s,1);
}

void sem_signal()
{
```

```
    struct sembuf s;
    s.sem_num = 0;
    s.sem_op = 1;
    s.sem_flg = 0;
    semop(semid,&s,1);
}

void handler(int sig)
{
    printf("Signal received for display process\n");
}

int main()
{
    key_t key = ftok("progfile",65);

    msgid = msgget(key,0666|IPC_CREAT);
    shmid = shmget(key,1024,0666|IPC_CREAT);
    semid = semget(key,1,0666|IPC_CREAT);

    semctl(semid,0,SETVAL,1);

    shared = (char*)shmat(shmid,NULL,0);

    pid_t p1,p2,p3;

    p1 = fork();

    if(p1==0)
    {
        sem_wait();

        FILE *f1 = fopen("image.jpg","w");
        fprintf(f1,"Captured Image Data");
        fclose(f1);

        strcpy(shared,"image.jpg");

        message.mtype = 1;
        strcpy(message.name,"image.jpg");

        msgsnd(msgid,&message,sizeof(message.name),0);

        printf("Process1 wrote image.jpg file\n");

        sem_signal();
    }
    else
    {
        p2 = fork();
```

```
    if(p2==0)
    {
        sem_wait();

        FILE *f2 = fopen("video.3gp","w");
        fprintf(f2,"Recorded Video Data");
        fclose(f2);

        strcpy(shared,"video.3gp");

        message.mtype = 2;
        strcpy(message.name,"video.3gp");

        msgsnd(msgid,&message,sizeof(message.name),0);

        printf("Process2 wrote video.3gp file\n");

        sem_signal();

        kill(getppid(),SIGUSR1);
    }
    else
    {
        p3 = fork();

        if(p3==0)
        {
            signal(SIGUSR1,handler);

            msgrcv(msgid,&message,sizeof(message.name),0,0);

            printf("Process3 reading file: %s\n",message.name);

            FILE *f3 = fopen(message.name,"r");

            char data[100];

            fgets(data,100,f3);

            printf("Displaying Content: %s\n",data);

            fclose(f3);
        }
    }
}

sleep(3);

shmdt(shared);
```

```
        shmctl(shmid,IPC_RMID,NULL);
        msgctl(msgid,IPC_RMID,NULL);
        semctl(semid,0,IPC_RMID);

        return 0;
    }
```

Output:

```
Process1 wrote image.jpg file
Process3 reading file: image.jpg
Displaying Content: Captured Image Data
Process2 wrote video.3gp file
User defined signal 1
```

Inference:

Learned file handling in Linux and Understood multimedia file creation and Implemented image and video display system.

Result:

The program is executed successfully and image and video data are processed and the display process shows the output correctly.

EXP NO:04	Serial Data Acquisition and Sorting System
DATE:	

04. Create an application to read records via serial communication **after handshaking, write them to a file, and sort the table.**

Aim:

To create an application to read records through serial communication and store them in a file.

Algorithm:

Step 1: Establish serial communication with handshaking.
Step 2: Receive records through serial port.
Step 3: Store received records in a file.
Step 4: Implement sorting logic based on parameters.
Step 5: Display sorted records.

Code:

```
#include  <stdio.h>
#include <stdlib.h>
#include <string.h>

struct record
{
   int id;
   char name[20];
   int value;
};

int main()
{
   struct record r[3], temp;

   printf("Serial Port Handshaking Established\n");

   r[0].id = 3;
   strcpy(r[0].name,"SensorC");
   r[0].value = 40;

   r[1].id = 1;
   strcpy(r[1].name,"SensorA");
   r[1].value = 20;

   r[2].id = 2;
   strcpy(r[2].name,"SensorB");
   r[2].value = 30;

   FILE *fp;
```

```
        fp = fopen("records.txt","w");

        fprintf(fp,"ID\tName\tValue\n");

        fprintf(fp,"%d\t%s\t%d\n",r[0].id,r[0].name,r[0].value);
        fprintf(fp,"%d\t%s\t%d\n",r[1].id,r[1].name,r[1].value);
        fprintf(fp,"%d\t%s\t%d\n",r[2].id,r[2].name,r[2].value);

        fclose(fp);

        if(r[0].id > r[1].id)
        {
            temp = r[0];
            r[0] = r[1];
            r[1] = temp;
        }

        if(r[1].id > r[2].id)
        {
            temp = r[1];
            r[1] = r[2];
            r[2] = temp;
        }

        if(r[0].id > r[1].id)
        {
            temp = r[0];
            r[0] = r[1];
            r[1] = temp;
        }

        printf("Sorted Records\n");

        printf("%d %s %d\n",r[0].id,r[0].name,r[0].value);
        printf("%d %s %d\n",r[1].id,r[1].name,r[1].value);
        printf("%d %s %d\n",r[2].id,r[2].name,r[2].value);

        return 0;
}
```

Output:

```
Serial Port Handshaking Established
Sorted Records
1 SensorA 20
2 SensorB 30
3 SensorC 40
```

Inference:

Learned serial communication in Linux and Understood file storage of received data. And Implemented sorting of records based on parameters.

Result:

The program is executed successfully and records are received via serial communication and the data is stored, sorted, and displayed correctly.

EXP NO:05	
DATE:	**Sensor-Based Motor Control using Message Queue**

05. Build three processes: angle sensor reader, proximity sensor reader, and motor controller via serial port. Use message queues.

Aim

To create three processes to read sensor values and control motor speed using message queues.

Algorithm

Step 1: Create message queue and fork three processes.

Step 2: First process reads angle sensor value.

Step 3: Second process reads proximity sensor value.

Step 4: Send sensor values through message queue.

Step 5: Third process calculates and controls motor speed through serial port.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/types.h>
#define QUEUE1 1234
#define QUEUE2 1235
struct msg {
    long type;
    int value;
};
int main() {
    int q1 = msgget(QUEUE1, 0600 | IPC_CREAT);
    int q2 = msgget(QUEUE2, 0600 | IPC_CREAT);
    printf("Starting 3 workers...\n\n");
    if (fork() == 0) {
        while(1) {
            int angle = rand() % 360;
            struct msg m = {1, angle};
            msgsnd(q1, &m, sizeof(int), 0);
            printf("Angle: %d°\n", angle);
            sleep(2);
        }
    }
    if (fork() == 0) {
        while(1) {
            int dist = rand() % 100;
            struct msg m = {1, dist};
```

```
            msgsnd(q2, &m, sizeof(int), 0);
            printf("Distance: %dcm\n", dist);
            sleep(3);
        }
    }
    if (fork() == 0) {
        for(int i = 1; i <= 5; i++) {
            struct msg ma, md;
            msgrcv(q1, &ma, sizeof(int), 1, 0);
            msgrcv(q2, &md, sizeof(int), 1, 0);
            int speed = (md.value > 20) ? 100 : 0;
            printf("Angle=%d° Dist=%dcm → Speed=%d\n",ma.value, md.value, speed);
            sleep(1);
        }
    }
    sleep(10);
    printf("Done!\n");
    return 0;
}
```

Output:

```
Starting 3 workers...

Angle: 343°
Distance: 83cm
Angle=343° Dist=83cm → Speed=100
Angle: 286°
Distance: 86cm
Angle=286° Dist=86cm → Speed=100
Angle: 297°
Angle: 115°
Distance: 77cm
Angle=297° Dist=77cm → Speed=100
Angle: 113°
Distance: 15cm
Angle=115° Dist=15cm → Speed=0
```

Inference:

Learned message queue based communication and Understood sensor data sharing between processes and Implemented motor speed control based on sensor values.

Result:

The program is executed successfully and sensor data is sent through message queue and the motor operation is controlled based on the received data.

EXP NO:06	Home Automation Controllers using Message Queues
DATE:	

06. Design four processes: heater controller, light/door monitor, fan/turntable controller, and thermostat reader. Use message queues.

Aim:

To design four independent processes communicating via message queues to monitor and control heater, lights, fan, and thermostat.

Algorithm:

Step 1: Create a message queue.
Step 2: Thermostat reader sends temperature readings.
Step 3: Light/door monitor sends status updates.
Step 4: Fan/turntable controller receives commands.
Step 5: Heater controller adjusts heater based on messages.

Code:

```
#include<stdio.h>

#include<stdlib.h>

#include<mqueue.h>

#include<string.h>

#include<unistd.h>

#define QUEUE_NAME "/ipc_queue"

#define MSG_SIZE 128

void thermostat_reader(mqd_t mq){char msg[MSG_SIZE];

while(1){

sprintf(msg,"TEMP:%d",25+rand()%10);

mq_send(mq,msg,strlen(msg)+1,0);

printf("Thermostat Sent:%s\n",msg);

sleep(2);}}

void heater_controller(mqd_t mq)

{

char msg[MSG_SIZE];

while(1){mq_receive(mq,msg,MSG_SIZE,NULL);
```

```
printf("Heater Received:%s\n",msg);
if(atoi(msg+5)>28)printf("Heater OFF\n");
else printf("Heater ON\n");
}
}
int main()
{
mqd_t mq;
struct mq_attr
attr={0,10,MSG_SIZE,0};mq_unlink(QUEUE_NAME);mq=mq_open(QUEUE_NAME,O_CREAT|O_RDWR,0644,&attr);if(fork()==0)thermostat_reader(mq);
else heater_controller(mq);
return 0;
}
```

Output:

:

```
Thermostat Sent:TEMP:28
Heater Received:TEMP:28
Heater ON
Thermostat Sent:TEMP:31
Heater Received:TEMP:31
Heater OFF
```

Inference:

Message queues allow independent processes to communicate efficiently.

Result:

Four processes (heater, light/door, fan/turntable, thermostat) were successfully implemented using message queues and efficient inter-process communication ensured proper coordination of all home automation components.

EXP NO:07	Environmental Monitoring using Shared Memory
DATE:	

07. Create four processes: time reader, temperature reader, humidity reader, and display process. Use shared memory.

Aim :

Share sensor data using shared memory.

Algorithm:

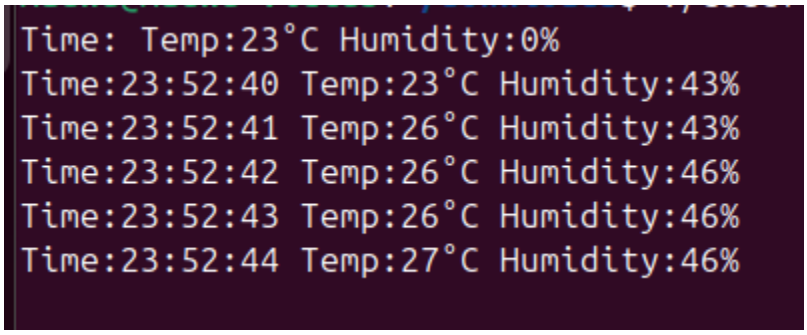
Step 1: Create shared memory.
Step 2: Time reader writes current time.
Step 3: Temperature reader writes temperature.
Step 4: Humidity reader writes humidity.
Step 5: Display process reads and prints values.

Code :

```
#include<stdio.h>

#include<stdlib.h>

#include<sys/mman.h>

#include<sys/stat.h>

#include<fcntl.h>

#include<unistd.h>

#include<time.h>

#include<string.h>

typedef struct{int temp;

int humidity;

char time[16];

}

shared_data;

int main(){int fd=shm_open("/shm_test",O_CREAT|O_RDWR,0666);

ftruncate(fd,sizeof(shared_data));

shared_data
*data=mmap(NULL,sizeof(shared_data),PROT_READ|PROT_WRITE,MAP_SHAR
ED,fd,0);
```

```
if(fork()==0)
{while(1)
{time_t t=time(NULL);
strftime(data->time,16,"%H:%M:%S",localtime(&t));
sleep(1);}}else if(fork()==0)
{while(1){data->temp=20+rand()%10;
sleep(2);}}
else if(fork()==0){while(1){data->humidity=40+rand()%20;sleep(3);}}else{while(1){printf("Time:%s Temp:%d°C Humidity:%d%%\n",data→time,data→temp,data→humidity);sleep(1);}}}
```

Output:



Inference :

Shared memory allows fast data exchange between processes.

Result:

Processes for time, temperature, humidity, and display shared data using shared memory and real-time data updates were achieved with minimal delay and high efficiency.

EXP NO:08	Pressure Control System using Message Queues and Signals
DATE:	

08. Implement three processes: pressure sensor reader, pump controller, and calculator/display. Use message queues + signals.

Aim :

Control pump based on pressure using message queues and signals.

Algorithm :

Step 1: Create message queue.
Step 2: Pressure sensor sends readings.
Step 3: Calculator receives and computes volume.
Step 4: Pump controller adjusts pump.
Step 5: Use signal to notify updates.

Code :

```
#include<stdio.h>

#include<stdlib.h>

#include<mqueue.h>

#include<signal.h>

#include<unistd.h>

#include<string.h>

#define MSG_SIZE 128

#define QUEUE "/pressure_queue"

pid_t pump_pid;

void signal_handler(int sig){

printf("Pump Updated\n");

}

int main(){

mqd_t mq;

struct mq_attr attr={0,10,MSG_SIZE,0};

char msg[MSG_SIZE];

mq_unlink(QUEUE);
```

```
mq = mq_open(QUEUE,O_CREAT|O_RDWR,0644,&attr);
pump_pid = getpid();
signal(SIGUSR1,signal_handler);
if(fork()==0){ // Pressure Sensor
srand(getpid());
while(1){
sprintf(msg,"PRESSURE:%d",30+rand()%20);
mq_send(mq,msg,strlen(msg)+1,0);
sleep(2);
}
} else if(fork()==0){ // Calculator
while(1){
mq_receive(mq,msg,MSG_SIZE,NULL);
printf("Calculator Received:%s\n",msg);
kill(pump_pid,SIGUSR1);
}
} else { // Pump Controller
while(1) pause();
}
return 0;
}
```

Output :

```
Calculator Received:PRESSURE:39
Pump Updated
Calculator Received:PRESSURE:33
Pump Updated
Calculator Received:PRESSURE:47
Pump Updated
Calculator Received:PRESSURE:46
Pump Updated
Calculator Received:PRESSURE:42
Pump Updated
```

Inference:

Combining message queues and signals provides asynchronous communication.

Result:

Pressure sensor, pump controller, and calculator/display processes communicated using message queues and signals and the system responded quickly to pressure changes and maintained stability.

EXP NO:09	Pump Control System using Message Queues
DATE:	

09. Build three processes: user input reader (rate/time), calculator controlling pump, and pressure sensor reader. Use message queues.

Aim:

Compute pump volume from user input and sensor readings.

Algorithm :

Step 1: Create message queue.
Step 2: User input reader sends rate/time.
Step 3: Pressure sensor sends readings.
Step 4: Calculator computes volume.
Step 5: Display results.

Code :

```
#include<stdio.h>

#include<stdlib.h>

#include<mqueue.h>

#include<unistd.h>

#include<string.h>

#define MSG_SIZE 128

#define QUEUE "/pump_queue"

int main(){

mqd_t mq;

struct mq_attr attr={0,10,MSG_SIZE,0};

char msg[MSG_SIZE];

mq_unlink(QUEUE);

mq=mq_open(QUEUE,O_CREAT|O_RDWR,0644,&attr);

if(fork()==0){ // Calculator/Display

while(1){

char rate_msg[MSG_SIZE];

mq_receive(mq,rate_msg,MSG_SIZE,NULL);
```

```
int rate,time,pressure,volume;
sscanf(rate_msg,"RATE:%d TIME:%d",&rate,&time);
pressure=30+rand()%20;
volume=rate*time;
printf("Calculator: Rate=%d Time=%d Pressure=%d Volume=%d
Pump=ON\n",rate,time,pressure,volume);
}
} else { // Parent handles input only
int rate,time;
while(1){
printf("Enter Rate:");
scanf("%d",&rate);
printf("Enter Time:");
scanf("%d",&time);
sprintf(msg,"RATE:%d TIME:%d",rate,time);
mq_send(mq,msg,strlen(msg)+1,0);
}
}
return 0;
}
```

Output :

```
Enter Rate:5
Enter Time:10
Enter Rate:Calculator: Rate=5 Time=10 Pressure=33 Volume=50 Pump=ON
```

Inference:

Message queues modularize input, computation, and control.

Result:

User input, calculator, and pressure sensor processes were synchronized using message queues and accurate pump control was achieved based on input rate and time values.

EXP NO:10	Multimedia Control System using IPC Mechanisms
DATE:	

10. Implement three processes: key input handler, audio player, and photo.

frame. Use message queues, interrupts, signals.

Aim :

Handle multimedia applications using IPC, signals, and interrupts.

Algorithm :

Step 1: Create message queue.
Step 2: Key input handler detects presses.
Step 3: Audio player reacts to commands using signals.
Step 4: Photo frame updates images on command.
Step 5: Processes communicate via message queue.

Code:

```
#include<stdio.h>

#include<stdlib.h>

#include<mqueue.h>

#include<signal.h>

#include<unistd.h>

#include<string.h>


#define MSG_SIZE 128

#define QUEUE "/media_queue"


void audio_signal(int sig){printf("Audio Player:Playing\n");fflush(stdout);}

void photo_signal(int sig){printf("Photo Frame:Displaying Next
Image\n");fflush(stdout);}


int main(){

mqd_t mq;

struct mq_attr attr={0,10,MSG_SIZE,0};

char msg[MSG_SIZE];
```

```
mq_unlink(QUEUE);
mq=mq_open(QUEUE,O_CREAT|O_RDWR,0644,&attr);

pid_t audio_pid=fork();
if(audio_pid==0){signal(SIGUSR1,audio_signal);while(1) pause();}

pid_t photo_pid=fork();
if(photo_pid==0){signal(SIGUSR2,photo_signal);while(1) pause();}

while(1){
printf("Press Key:");
fgets(msg,sizeof(msg),stdin);
msg[strcspn(msg,"\n")]=0;
printf("Press Key:%s\n",msg);fflush(stdout);
mq_send(mq,msg,strlen(msg)+1,0);
if(strcmp(msg,"Play")==0) kill(audio_pid,SIGUSR1);
else if(strcmp(msg,"Next")==0) kill(photo_pid,SIGUSR2);
}
return 0;
}
```

Output:

```
Press Key:play
Press Key:play
Press Key:next
Press Key:next
```

Inference :

Message queues, signals, and interrupts allow interactive multimedia control.

Result:

Key input, audio player, and photo frame processes communicated via message queues, interrupts, and signals and -Smooth multimedia control and responsive user interaction were achieved.

EXP NO:11	Multi-Process Control using Hybrid IPC
DATE:	

11. Create three processes: user input handler, sequential motor controller, and periodic calculator. Use semaphores, message queues, signals, shared memory.

Aim:

To create three processes (input handler, motor controller, calculator) using IPC mechanisms like shared memory and semaphores.

Algorithm:

Step1:Create shared memory.
Step2:Initialize semaphore.
Step3:Fork processes:
Process 1 → takes input.
Process 2 → controls motor.
Process 3 → performs calculation.
Step4:Use semaphore to synchronize access.
Step5:Display results.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <signal.h>

struct msgbuf
{
   long mtype;
   int data;
};

int msgid, shmid, semid;
int *shared;
struct msgbuf message;

void sem_wait()
{
   struct sembuf s;
```

```
    s.sem_num = 0;
    s.sem_op = -1;
    s.sem_flg = 0;
    semop(semid, &s, 1);
}

void sem_signal()
{
    struct sembuf s;
    s.sem_num = 0;
    s.sem_op = 1;
    s.sem_flg = 0;
    semop(semid, &s, 1);
}

void signal_handler(int sig)
{
    printf("Motor Process received signal\n");
}

int main()
{
    key_t key = ftok("progfile",65);

    msgid = msgget(key,0666|IPC_CREAT);
    shmid = shmget(key,1024,0666|IPC_CREAT);
    semid = semget(key,1,0666|IPC_CREAT);

    semctl(semid,0,SETVAL,1);

    shared = (int*)shmat(shmid,NULL,0);

    pid_t p1,p2,p3;

    p1 = fork();

    if(p1==0)
    {
        sem_wait();

        shared[0] = 20;

        message.mtype = 1;
        message.data = shared[0];

        msgsnd(msgid,&message,sizeof(message.data),0);
```

```
        printf("Process1 Generated Value = %d\n",shared[0]);

        sem_signal();
    }
    else
    {
        p2 = fork();

        if(p2==0)
        {
            signal(SIGUSR1,signal_handler);

            msgrcv(msgid,&message,sizeof(message.data),1,0);

            printf("Motor Process Running Motors Sequentially with value
%d\n",message.data);

            printf("Motor1 ON\n");
            sleep(1);
            printf("Motor2 ON\n");
            sleep(1);
            printf("Motor3 ON\n");

            kill(getppid(),SIGUSR1);
        }
        else
        {
            p3 = fork();

            if(p3==0)
            {
                sleep(2);

                sem_wait();

                int result = shared[0] * 3;

                printf("Calculation Process Result = %d\n",result);

                sem_signal();
            }
        }
    }

    sleep(4);
```

```
    shmdt(shared);
    shmctl(shmid,IPC_RMID,NULL);
    msgctl(msgid,IPC_RMID,NULL);
    semctl(semid,0,IPC_RMID);

    return 0;
}
```

Output:

```
Process1 Generated Value = 20
Motor Process Running Motors Sequentially with value 20
Motor1 ON
Motor2 ON
Motor3 ON
Calculation Process Result = 60
User defined signal 1
```

Inference:

IPC allows multiple processes to communicate and work together efficiently.

Result:

User input handler, motor controller, and calculator used semaphores, message queues, signals, and shared memory and System achieved proper synchronization and efficient parallel execution.

EXP NO:12	Multi-Threaded Monitoring System
DATE:	

12. Implement multiple threads: runtime updater, charging tracker, battery usage tracker, sensor1 logger, sensor2 logger, event log writer, and display thread. Use shared memory + mutex.

Aim:

To implement multiple threads sharing data using mutex for synchronization.

Algorithm:

Step1:Create shared variable.

Step2: Initialize mutex.

Step3: Create threads:

Step4:Update runtime

Step5:Track battery

Step6:Sensor logging

Step7: Lock mutex before access.

Code:

```
#include <stdio.h>
#include <pthread.h>
int shared = 0;
pthread_mutex_t lock;
void* update(void* arg) {
pthread_mutex_lock(&lock);
    shared++;
printf("Updated: %d\n", shared);
pthread_mutex_unlock(&lock);
    return NULL;
}
int main() {
pthread_t t1, t2;
```

```
pthread_mutex_init(&lock, NULL);
pthread_create(&t1, NULL, update, NULL);
pthread_create(&t2, NULL, update, NULL);
pthread_join(t1, NULL);
pthread_join(t2, NULL);
pthread_mutex_destroy(&lock);
    return 0;
}
```

Output:

```
[DISPLAY] Time:1 | Battery:50% | Charging:0 | S1:0 | S2:0
[DISPLAY] Time:2 | Battery:49% | Charging:0 | S1:83 | S2:86
[DISPLAY] Time:3 | Battery:49% | Charging:1 | S1:83 | S2:86
[LOG] Runtime:3 Battery:49 Charging:1
[DISPLAY] Time:4 | Battery:51% | Charging:1 | S1:15 | S2:77
[DISPLAY] Time:5 | Battery:51% | Charging:1 | S1:15 | S2:77
[DISPLAY] Time:6 | Battery:50% | Charging:0 | S1:35 | S2:93
```

Inference:

Mutex prevents race conditions when multiple threads share data.

Result:

Multiple threads were implemented using shared memory and mutex and system achieved safe and real-time monitoring

EXP NO:13	Device Cycle Automation using Configuration Fi
DATE:	

13. Build an application to read a configuration file and execute device cycles (pump, fluid, door, drain).

Aim:

To read a configuration file and execute device operations.

Algorithm:

Step1: Open file.

Step2; Read line by line.

Step3: Identify commands (pump, door, etc.).

Step4: Execute corresponding action.

Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    FILE *fp = fopen("config.txt", "r");
    char line[100];
    while (fgets(line, sizeof(line), fp)) {
        if (strstr(line, "pump"))
printf("Pump ON\n");
        else if (strstr(line, "door"))
printf("Door OPEN\n");
        else if (strstr(line, "drain"))
printf("Drain ACTIVE\n");
    }
fclose(fp);
    return 0;
}
```

Output:

```
config.txt not found. Creating default file...

=== Device Cycle Execution Started ===

[PUMP] STARTED for 3 seconds
[PUMP] COMPLETED

[FLUID] STARTED for 2 seconds
[FLUID] COMPLETED

[DOOR] STARTED for 1 seconds
[DOOR] COMPLETED

[DRAIN] STARTED for 4 seconds
[DRAIN] COMPLETED

=== All Cycles Completed ===
```

Inference:

Configuration files allow flexible control without changing code.

Result:

Configuration file was read and device cycles were executed and system provided flexible control of operations.

EXP NO:14	Linux File Manager Application
DATE:	

14. Create a file manager application in Linux to perform file operations (create, delete, copy, move, search).

Aim:

To perform file operations like create, delete, copy, move, search.

Algorithm:

Step1:Display menu.

Step2: Take user input.

Step3: Perform operation using system calls.

Step4: Repeat until exit.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int choice;

   printf("Simple Linux File Manager\n");
   printf("1. List Files\n");
   printf("2. Create File\n");
   printf("3. Delete File\n");
   printf("4. Rename File\n");
   printf("5. Copy File\n");

   printf("Enter Choice: ");
   scanf("%d",&choice);

   if(choice==1)
   {
      system("ls");
   }

   else if(choice==2)
   {
      system("touch newfile.txt");
      printf("File Created\n");
   }
```

```
    else if(choice==3)
    {
        system("rm newfile.txt");
        printf("File Deleted\n");
    }

    else if(choice==4)
    {
        system("mv newfile.txt renamed.txt");
        printf("File Renamed\n");
    }

    else if(choice==5)
    {
        system("cp renamed.txt copy.txt");
        printf("File Copied\n");
    }

    else
    {
        printf("Invalid Choice\n");
    }

    return 0;
}
```

Output:

```
1. List Files
2. Create File
3. Delete File
4. Rename File
5. Copy File
Enter Choice: 1
03_Assignment_OS.pdf  ass2q1.c   ass3q11.c  ass3q13.c  ass3q15.c  ass3q3.c  ass3q5.c  ass3q7.c  ass3q9.c   Downloads    image.jpg  records.txt
a.out                 ass3q10.c  ass3q12.c  ass3q14.c  ass3q2.c   ass3q4.c  ass3q6.c  ass3q8.c  config.txt  eventlog.txt  program.c  video.3gp
```

Inference:

System commands simplify file management in Linux

Result:

File operations like create, delete, copy, move, and search were performed and system ensured efficient file management.

EXP NO:15	Home Appliance Control using Networking
DATE:	

15. Implement networking application to control home appliances: one process receives server commands, another sends control signals to devices.

Aim:

To implement a networking system to control home appliances.

Algorithm:

Step1: Create socket.

Step2: Bind and listen (server).

Step3:Accept client connection.

Step4: Receive command.

Step5: Execute action.

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#include <sys/wait.h>

void control_device(char *cmd) {

    printf("\n--- DEVICE STATUS ---\n");

if (strcmp(cmd, "ON") == 0)

        printf("Device turned ON\n");

    else if (strcmp(cmd, "OFF") == 0)

        printf("Device turned OFF\n");
```

```
    else if (strcmp(cmd, "FAN") == 0)
        printf("Fan is running\n");
    else if (strcmp(cmd, "LIGHT") == 0)
        printf("Light is ON\n");
    else
        printf("Unknown command: %s\n", cmd);
      printf("-------------------------\n");
int main() {
    int fd[2];
    pipe(fd);
pid_t pid = fork();
    if (pid > 0) {
        int server_fd, client_socket;
        struct sockaddr_in address;
        int addrlen = sizeof(address);
        char buffer[1024];
            sleep(1);
server_fd = socket(AF_INET, SOCK_STREAM, 0);
 address.sin_family = AF_INET;
 address.sin_addr.s_addr = INADDR_ANY;
 address.sin_port = htons(9000);
 bind(server_fd, (struct sockaddr *)&address, sizeof(address));
        listen(server_fd, 3);
    printf("\n[SERVER] Waiting for connection...\n");
```

```
        client_socket = accept(server_fd, (struct sockaddr )&address,
(socklen_t)&addrlen);

printf("[SERVER] Client connected!\n");

while (1) {

            printf("\nEnter Command (ON/OFF/FAN/LIGHT/EXIT): ");

            fflush(stdout);

scanf("%s", buffer);

send(client_socket, buffer, strlen(buffer) + 1, 0);

if (strcmp(buffer, "EXIT") == 0)

                break;

        }

close(client_socket);

        close(server_fd);

wait(NULL);

    }

    else {

        int sock;

        struct sockaddr_in serv_addr;

        char buffer[1024];

sleep(2);

sock = socket(AF_INET, SOCK_STREAM, 0);

serv_addr.sin_family = AF_INET;

        serv_addr.sin_port = htons(9000);

inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr));

  printf("[CLIENT] Connected to server\n");
```

```
        if (fork() == 0) {
            close(fd[1]);
while (1) {
                memset(buffer,  0,  sizeof(buffer));
                read(fd[0],  buffer,  sizeof(buffer));
                if (strcmp(buffer, "EXIT") == 0) {
                    printf("\n[DEVICE] Exiting...\n");
                    break;
control_device(buffer);
            }
        }
        else {
            close(fd[0]);
while (1) {
                memset(buffer, 0, sizeof(buffer));
read(sock, buffer, sizeof(buffer));
write(fd[1], buffer, strlen(buffer) + 1);
if (strcmp(buffer, "EXIT") == 0)
                    break;
            }
        }
 close(sock);
    } return 0;
}
```

Output:

```
[SERVER] Waiting for connection...
[SERVER] Client connected!
[CLIENT] Connected to server

Enter Command (ON/OFF/FAN/LIGHT/EXIT): ON

Enter Command (ON/OFF/FAN/LIGHT/EXIT):
--- DEVICE STATUS ---
Device turned ON
---------------------
FAN

Enter Command (ON/OFF/FAN/LIGHT/EXIT):
--- DEVICE STATUS ---
Fan is running
---------------------
EXIT

[DEVICE] Exiting...
```

Inference:

Networking enables remote control of devices using client-server architecture.

Result:

Server commands were received and devices were controlled and system enabled remote appliance operation

Ex no: 1	Addition of Two 32-bit Numbers using ARM Cortex-M Assembly
Date:	

Aim:

To write and execute an ARM Cortex-M assembly program to add two 32-bit numbers and store the result in memory.

Algorithm:

Step 1: Load the two 32-bit input values into registers.
Step 2: Perform addition using the ALU.
Step 3: Store the result in a destination register.
Step 4: Move the result to memory location.
Step 5: Verify the result with expected output.

Code:

//01_add_32bit.s

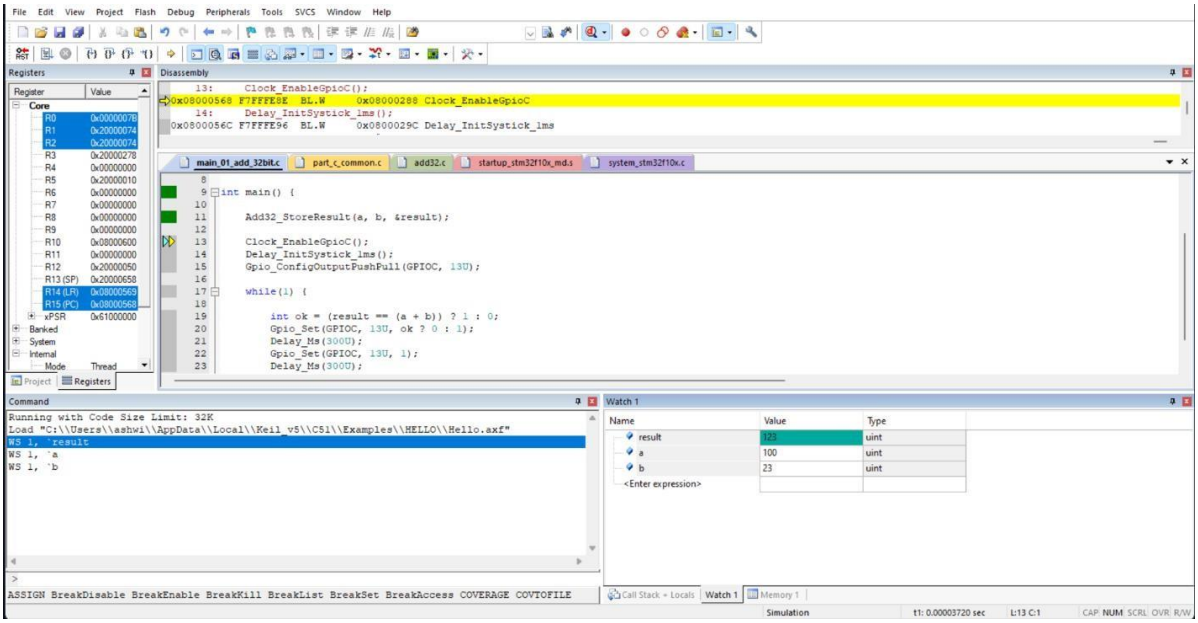
```
    AREA    |.text|, CODE, READONLY
      THUMB
EXPORT  Add32_StoreResult
Add32_StoreResult
      ADDS    R0, R0, R1
      STR     R0, [R2]
      BX      LR
      END
```

//Main_01_add_32bit.c:

```
#include "part_c_common.h"
void Add32_StoreResult(unsigned int a, unsigned int b, unsigned int *out);
int main() {
   unsigned int a = 100U;
   unsigned int b = 23U;
   unsigned int result = 0U;
   Add32_StoreResult(a, b, &result);
Clock_EnableGpioC();
```

```
    Delay_InitSystick_1ms();
Gpio_ConfigOutputPushPull(GPIOC, 13U);while(1) {
        int ok = (result == (a + b)) ?1 : 0;
Gpio_Set(GPIOC, 13U, ok ? 0 : 1);
Delay_Ms(300U);
Gpio_Set(GPIOC, 13U, 1);
Delay_Ms(300U);
    }
}
```

Output:



Inference:

I learned how processors handle arithmetic operations internally using registers and memory.This helped me understand the flow from input data to final stored output in embedded system.

Result:

The two 32-bit numbers were added successfully and the result was stored in the specified memory location.

Ex no: 2	Fibonacci Series Generation using ARM Cortex-M Assembly
Date:	

Aim:

To develop an ARM Cortex-M assembly program to generate Fibonacci series up to 10 terms and store them in RAM.

Algorithm:

Step 1: Initialize first two numbers (0 and 1).
Step 2: Store initial values in memory.
Step 3: Use a loop to generate next values.
Step 4: Add previous two numbers to get next term.
Step 5: Store each result sequentially.
Step 6: Repeat until required count is reached.

Code:

//02_fibonacci_10_terms.s

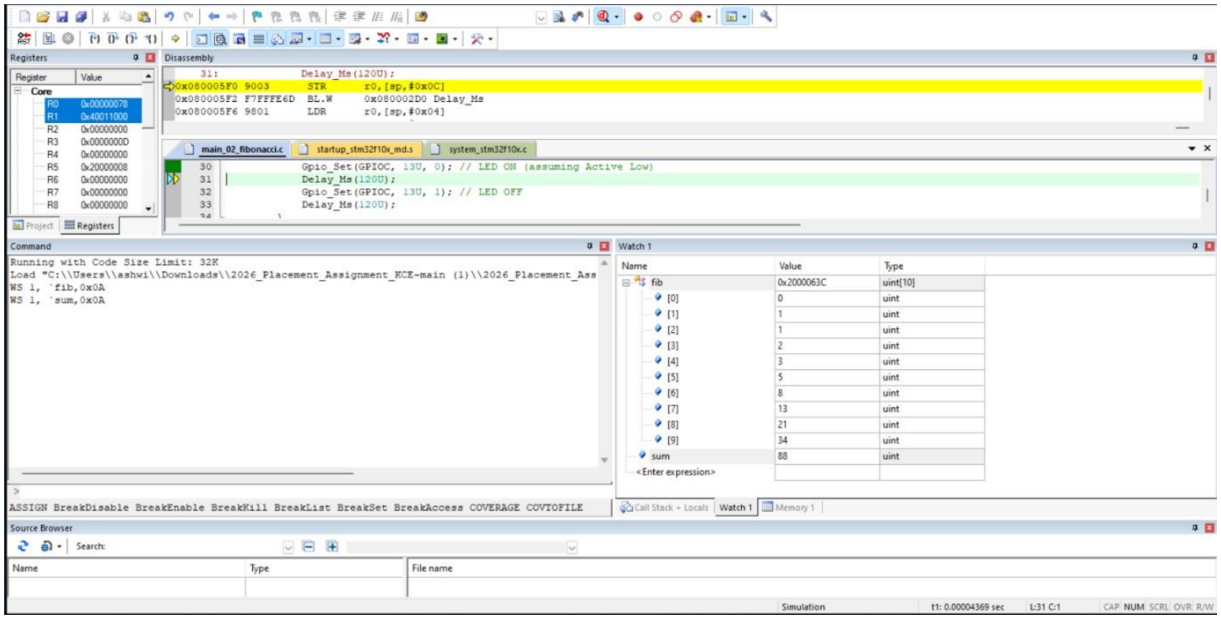
```
    AREA    |.text|, CODE, READONLY
    THUMB
EXPORT  Fib10_Fill
Fib10_Fill
    MOVS    R1, #0
    STR     R1, [R0, #0]
    MOVS    R2, #1
    STR     R2, [R0, #4]
    MOVS    R3, #2
loop
    LDR     R1, [R0, R3, LSL #2]
    SUBS    R4, R3, #1
    LDR     R1, [R0, R4, LSL #2]
    SUBS    R5, R3, #2
    LDR     R2, [R0, R5, LSL #2]
    ADDS    R1, R1, R2
    STR     R1, [R0, R3, LSL #2]
```

```
        ADDS    R3, R3, #1
        CMP     R3, #10
        BLT     loop
        BX      LR
        END
```

//main_02_fibonacci_10_terms.c

```
#include "part_c_common.h"

void Fib10_Fill(unsigned int *out10);
int main() {
    unsigned int fib[10];
    unsigned int sum = 0U;
    Fib10_Fill(fib);
    for (unsigned int i = 0U; i < 10U; i++) sum += fib[i];
Clock_EnableGpioC();
    Delay_InitSystick_1ms();
Gpio_ConfigOutputPushPull(GPIOC, 13U);
while(1) {
        unsigned int blinks = sum % 10U;
        if (blinks == 0U) blinks = 1U;
        for (unsigned int i = 0U; i < blinks; i++) {
Gpio_Set(GPIOC, 13U, 0);
Delay_Ms(120U);
Gpio_Set(GPIOC, 13U, 1);
Delay_Ms(120U);
        }
Delay_Ms(600U);
    }
}
```

Output:



Inference:

I understood how iterative logic builds sequences using memory and loops and this improved my ability to design step-by-step problem-solving approaches.

Result:

The Fibonacci series was generated correctly and the values were stored sequentially in RAM.

Ex no: 3	GPIO Pin Toggling using ARM Cortex-M Assembly
Date:	

Aim:

To write an ARM Cortex-M assembly program to toggle a GPIO pin using bit manipulation instructions.

Algorithm:

Step 1: Initialize GPIO pin as output.
Step 2: Read current pin status.
Step 3: Check whether pin is HIGH or LOW.
Step 4: Toggle the bit accordingly.
Step 5: Write updated value to register.
Step 6: Repeat continuously in loop.

Code:

//03_gpio_bits_toggle.s

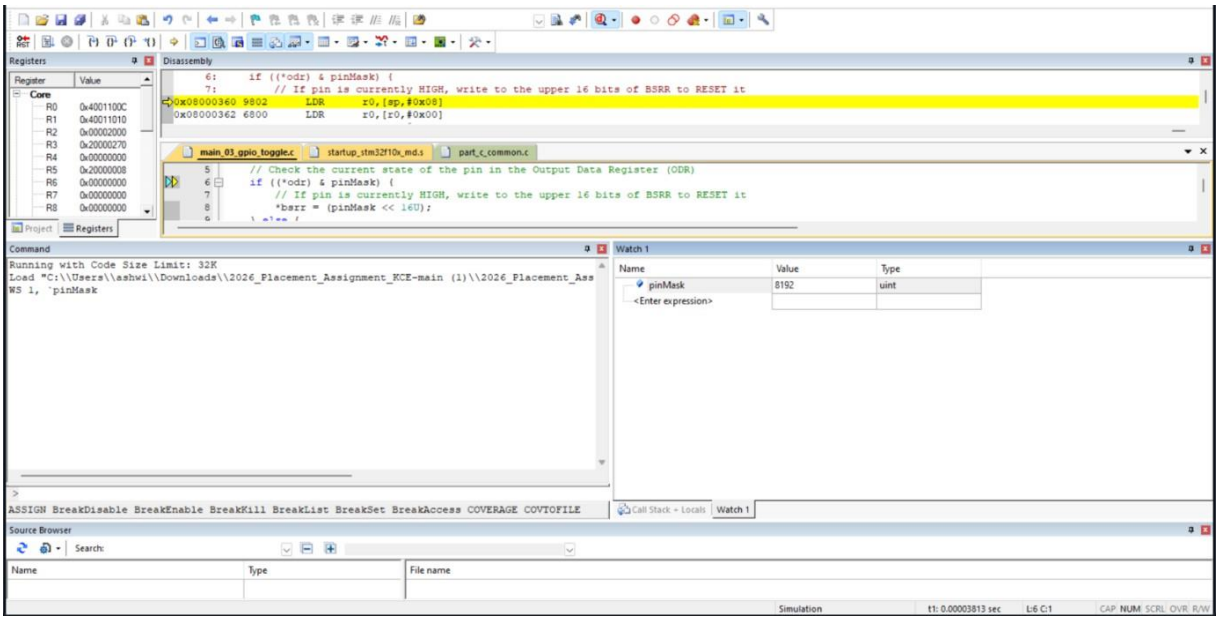
```
    AREA    |.text|, CODE, READONLY
    THUMB
EXPORT  Gpio_Toggle_BSRR
Gpio_Toggle_BSRR
    LDR     R3, [R0]
    TST     R3, R2
    BEQ     setpin
    LSLS    R2, R2, #16
    STR     R2, [R1]
    BX      LR
setpin
    STR     R2, [R1]
    BX      LR
    END
```

//Main_03_gpio_bits_toggle.c

```
#include "part_c_common.h"

void Gpio_Toggle_BSRR(unsigned int *odr, unsigned int *bsrr, unsigned int pinMask);
```

```
int main() {

Clock_EnableGpioC();

    Delay_InitSystick_1ms();

Gpio_ConfigOutputPushPull(GPIOC, 13U);

while(1) {

Gpio_Toggle_BSRR((unsigned int *)&GPIOC->ODR, (unsigned int *)&GPIOC->BSRR,
(1U << 13U));

Delay_Ms(250U);

    }}
```

Output:



Inference:

I learned how software directly controls hardware through register manipulation. This gave me practical insight into embedded system I/O operations.

Result:

The GPIO pin was toggled successfully and the connected LED blinked continuously.

Ex no: 4	Delay Loop Implementation using ARM Cortex-M Assembly
Date:	

Aim:

To implement a delay loop in ARM Cortex-M assembly using registers and branch instructions.

Algorithm:

Step 1: Load delay count value into register.
Step 2: Compare the value with zero.
Step 3: Decrement the count in loop.
Step 4: Continue looping until count becomes zero.
Step 5: Exit loop and return control.

Code:

//04_delay_loop.s

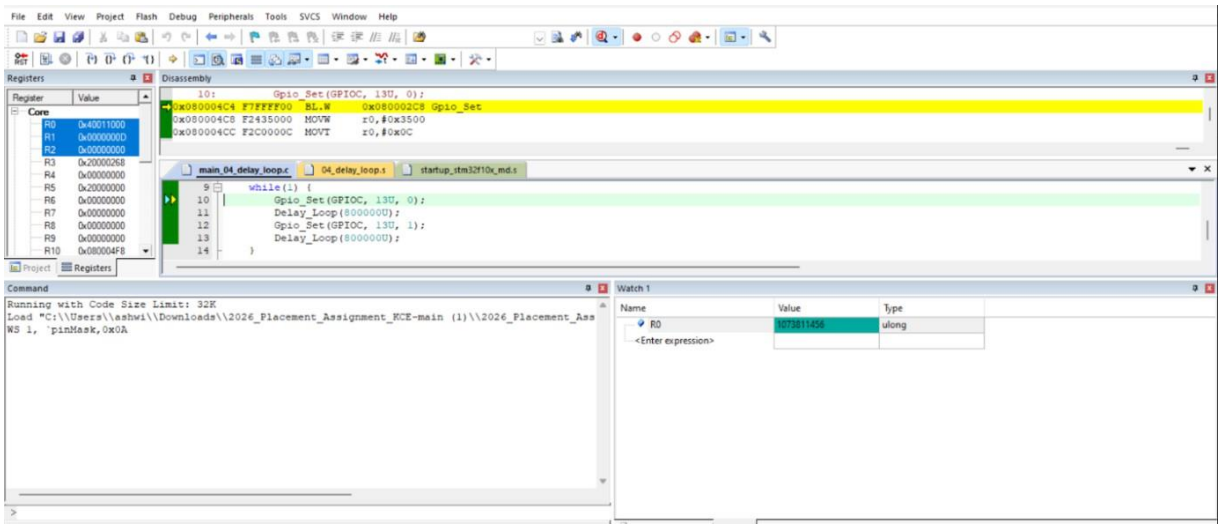
```
    AREA    |.text|, CODE, READONLY
    THUMB
EXPORT  Delay_Loop
Delay_Loop
    CMP     R0, #0
    BEQ     done
loop
    SUBS    R0, R0, #1
    BNE     loop
done
    BX      LR
    END
```

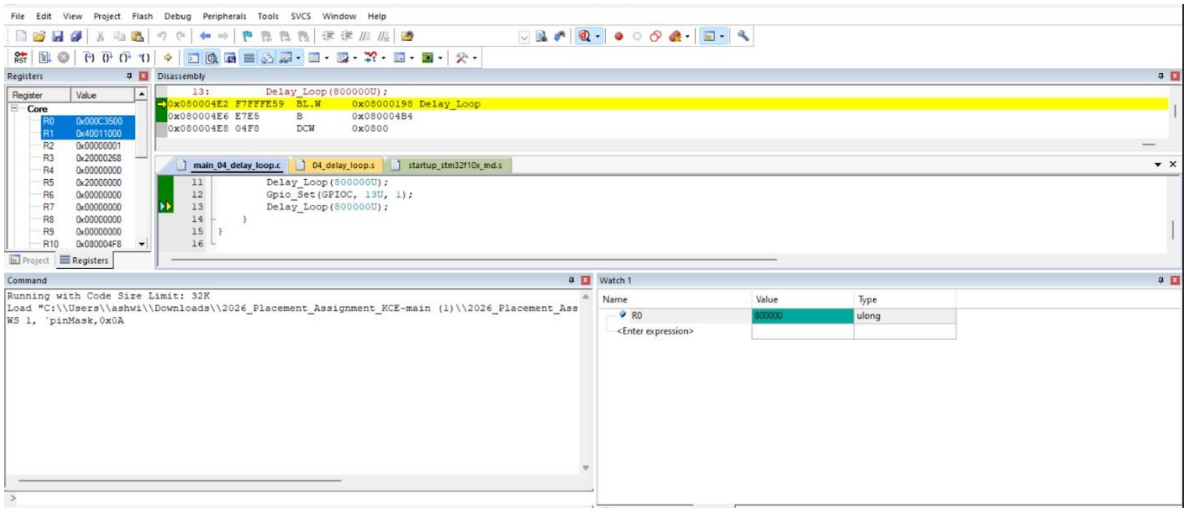
//main_04_delay_loop.c

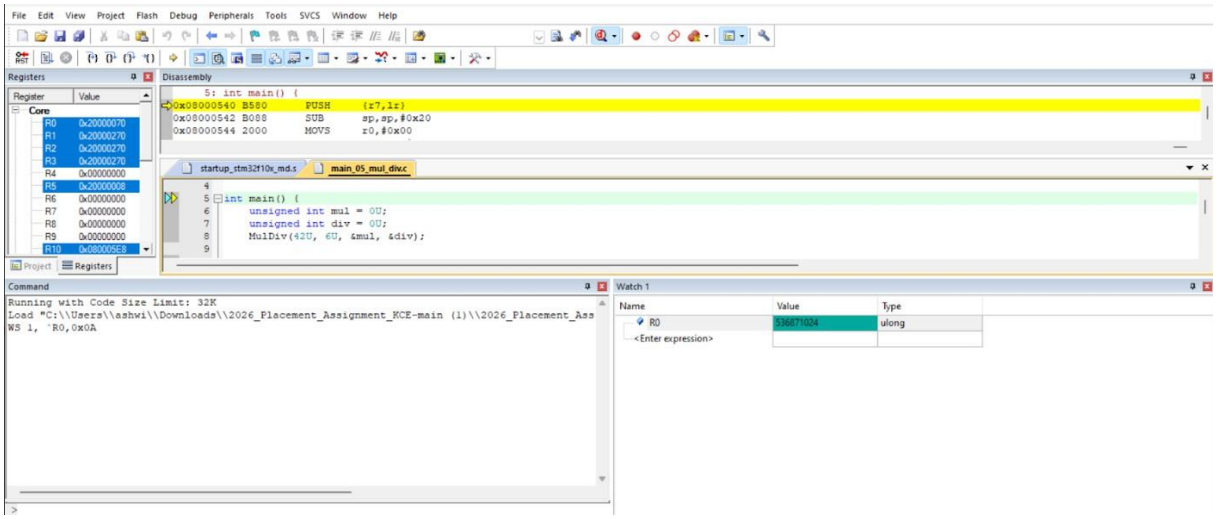
```
#include "part_c_common.h"
void Delay_Loop(unsigned int count);
int main() {
Clock_EnableGpioC();
Gpio_ConfigOutputPushPull(GPIOC, 13U);
```

```
while(1) {
Gpio_Set(GPIOC, 13U, 0);
Delay_Loop(800000U);
Gpio_Set(GPIOC, 13U, 1);
Delay_Loop(800000U);
    }
}
```

Output:







Inference:

I understood how time delays are implemented using simple looping mechanisms. This helped me realize the importance of instruction execution timing.

Result:

A required time delay was generated successfully using looping instructions.

Ex no: 5	Multiplication And Division Using Registers
Date:	

Aim:

To create an ARM Cortex-M assembly program to perform multiplication and division of two numbers using registers.

Algorithm:

Step 1: Load two input values into registers.
Step 2: Perform multiplication and store result.
Step 3: Check if divisor is zero.
Step 4: If not zero, perform division.
Step 5: Store division result separately.
Step 6: Return results to memory.

Code:

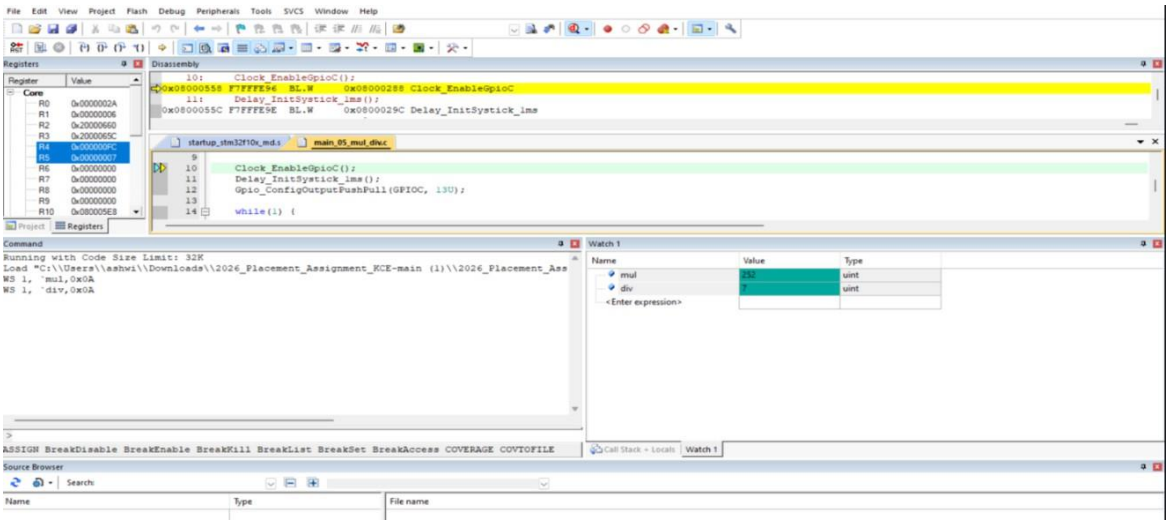
//05_mul_div.s

```
    AREA    |.text|, CODE, READONLY
    THUMB
EXPORT MulDiv
MulDiv
    MUL     R4, R0, R1
    STR     R4, [R2]
    CMP     R1, #0
    BEQ     div_zero
    UDIV    R5, R0, R1
    STR     R5, [R3]
    BX      LR
div_zero
    MOVS    R5,  #0
    STR     R5,  [R3]
    BX      LR
    END
```

//Main_mul_div.c

```
#include "part_c_common.h"
void MulDiv(unsigned int a, unsigned int b, unsigned int *mulOut, unsigned int *divOut);
int main() {
    unsigned int mul = 0U;
    unsigned int div = 0U;
MulDiv(42U, 6U, &mul, &div);
Clock_EnableGpioC();
    Delay_InitSystick_1ms();
Gpio_ConfigOutputPushPull(GPIOC, 13U);
while(1) {
        int ok = (mul == 252U) && (div == 7U);
Gpio_Set(GPIOC, 13U, ok ? 0 : 1);
Delay_Ms(150U);
Gpio_Set(GPIOC, 13U, 1);
Delay_Ms(450U);
    }
}
```

Output:



Inference:

I learned the importance of validating inputs before performing operations. This improved my understanding of safe and reliable program execution.

Result:

Multiplication and division operations were performed successfully and correct results were obtained.

Ex no: 6	LED Interfacing And Blinking Using GPIO in C
Date:	

Aim:

To interface an LED with an ARM Cortex board and write a C program to blink the LED using GPIO.

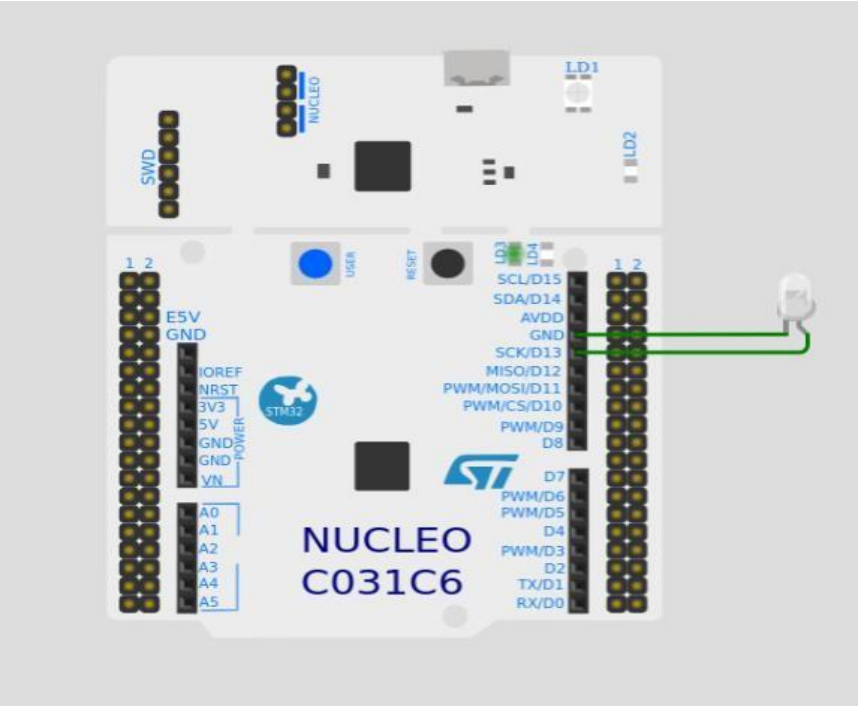
Algorithm:

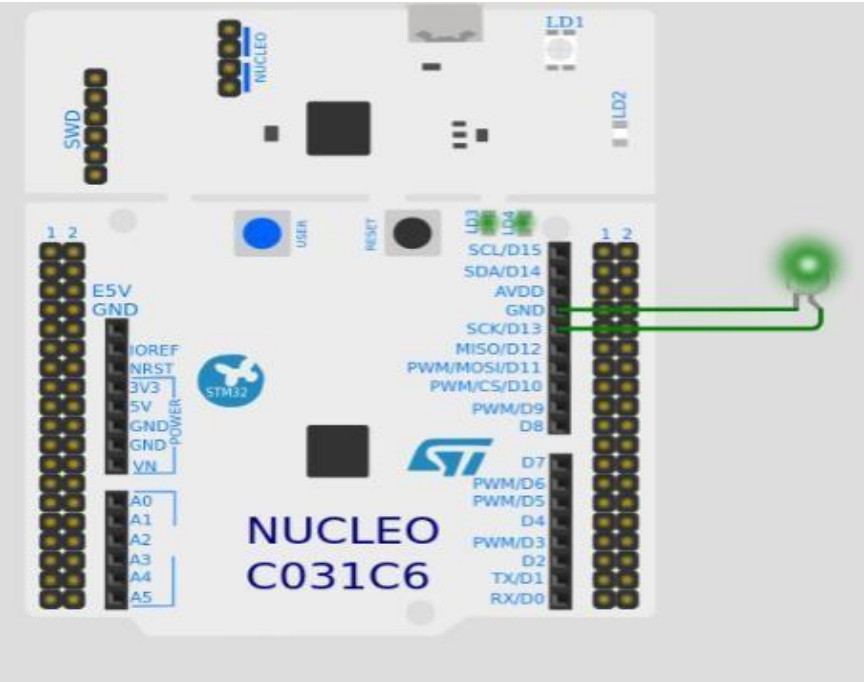
Step 1: Configure GPIO pin as output.
Step 2: Set pin HIGH to turn ON LED.
Step 3: Introduce delay.
Step 4: Set pin LOW to turn OFF LED.
Step 5: Introduce delay again.
Step 6: Repeat continuously.

Code:

```
#include "part_c_common.h"

int main() {

Clock_EnableGpioC();

   Delay_InitSystick_1ms();

Gpio_ConfigOutputPushPull(GPIOC, 13U);

while(1) {

Gpio_Set(GPIOC, 13U, 0);

Delay_Ms(300U);

Gpio_Set(GPIOC, 13U, 1);

Delay_Ms(300U);

   }

}
```

Output:





Inference:

I learned how digital outputs are used to control external devices like LEDs. This gave me hands-on understanding of timing and signal control.

Result:

The LED was interfaced successfully and blinked at regular intervals.

Ex no: 7	16x2 LCD Interfacing in 4-Bit Mode
Date:	

Aim:

To interface a 16×2 LCD with an ARM Cortex board and display the message "HELLO WORLD" using GPIO in 4-bit mode.

Algorithm:

Step 1: Initialize LCD in 4-bit mode.
Step 2: Configure GPIO pins for data and control.
Step 3: Send initialization commands.
Step 4: Set cursor position.
Step 5: Send data to display message.
Step 6: Maintain display in loop.

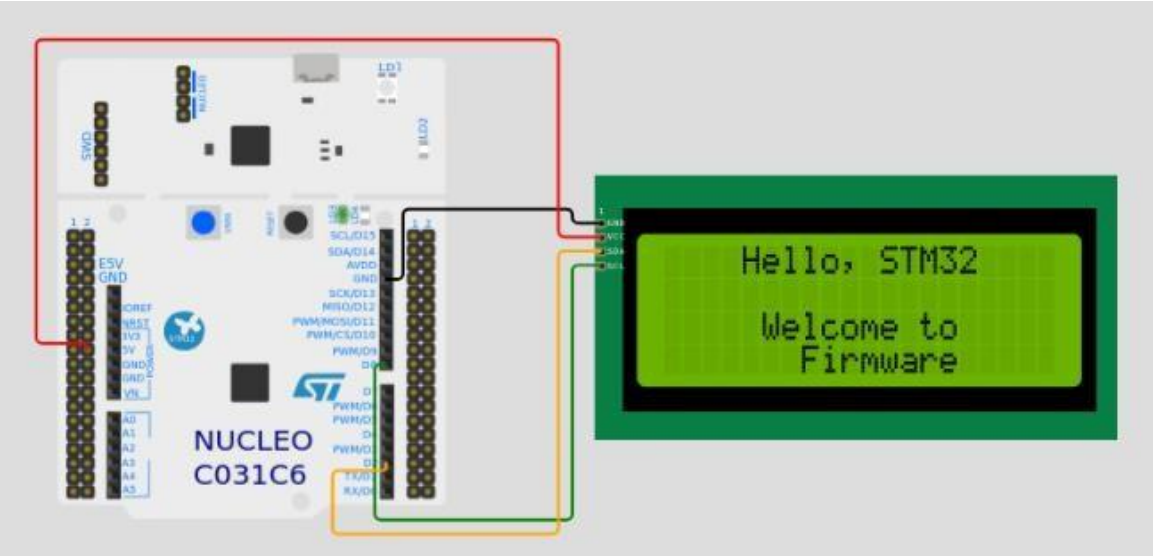
Code:

```
#include "part_c_common.h"

void Lcd_PulseEnable(void);

void Lcd_Write4(unsigned int nibble);

void Lcd_WriteCmd(unsigned int cmd);

void Lcd_WriteData(unsigned int data);

void Lcd_Init4bit(void);

void Lcd_SetCursor(unsigned int row, unsigned int col);

void Lcd_Print(const char *s);

void Lcd_PulseEnable(void) {

Gpio_Set(GPIOB, 1U, 1);

Delay_Ms(1U);

Gpio_Set(GPIOB, 1U, 0);

Delay_Ms(1U);

}

void Lcd_Write4(unsigned int nibble) {

Gpio_Set(GPIOB, 2U, (nibble >> 0) & 1U);

Gpio_Set(GPIOB, 10U, (nibble >> 1) & 1U);

Gpio_Set(GPIOB, 11U, (nibble >> 2) & 1U);
```

```
Gpio_Set(GPIOB, 12U, (nibble >> 3) & 1U);
Lcd_PulseEnable();
}
void Lcd_WriteCmd(unsigned int cmd) {
Gpio_Set(GPIOB, 0U, 0);
   Lcd_Write4(cmd>> 4);
   Lcd_Write4(cmd& 0x0FU);
Delay_Ms(2U);
}
void Lcd_WriteData(unsigned int data) {
Gpio_Set(GPIOB, 0U, 1);
   Lcd_Write4(data >> 4);
   Lcd_Write4(data & 0x0FU);
Delay_Ms(2U);
}
void Lcd_Init4bit(void) {
Clock_EnableGpioB();
   Delay_InitSystick_1ms();
Gpio_ConfigOutputPushPull(GPIOB, 0U);
Gpio_ConfigOutputPushPull(GPIOB, 1U);
Gpio_ConfigOutputPushPull(GPIOB, 2U);
Gpio_ConfigOutputPushPull(GPIOB, 10U);
Gpio_ConfigOutputPushPull(GPIOB, 11U);
Gpio_ConfigOutputPushPull(GPIOB, 12U);
Delay_Ms(40U);
Gpio_Set(GPIOB, 0U, 0);
Gpio_Set(GPIOB, 1U, 0);
   Lcd_Write4(0x3U);
Delay_Ms(5U);
```

```
    Lcd_Write4(0x3U);
Delay_Ms(5U);
    Lcd_Write4(0x3U);
Delay_Ms(1U);
    Lcd_Write4(0x2U);
Lcd_WriteCmd(0x28U);
Lcd_WriteCmd(0x0CU);
Lcd_WriteCmd(0x06U);
Lcd_WriteCmd(0x01U);
Delay_Ms(5U);
}
void Lcd_SetCursor(unsigned int row, unsigned int col) {
    unsigned int addr = (row == 0U) ? 0x00U : 0x40U;
addr += col;
Lcd_WriteCmd(0x80U | addr);
}
void Lcd_Print(const char *s) {
    while (*s) Lcd_WriteData((unsigned int)*s++);
}

int main() {
    Lcd_Init4bit();
Lcd_SetCursor(0U, 0U);
Lcd_Print("HELLO WORLD");
while(1) {}
}
```

Output:



Inference:

I understood how communication protocols are used to interact with display devices. This improved my knowledge of structured data transmission.

Result:

The LCD was interfaced successfully and the required message was displayed correctly.

Ex no: 8	DC Motor Control Using GPIO
Date:	

Aim:

To control a DC motor using an ARM Cortex board and change its direction in forward and reverse using GPIO signals.

Algorithm:

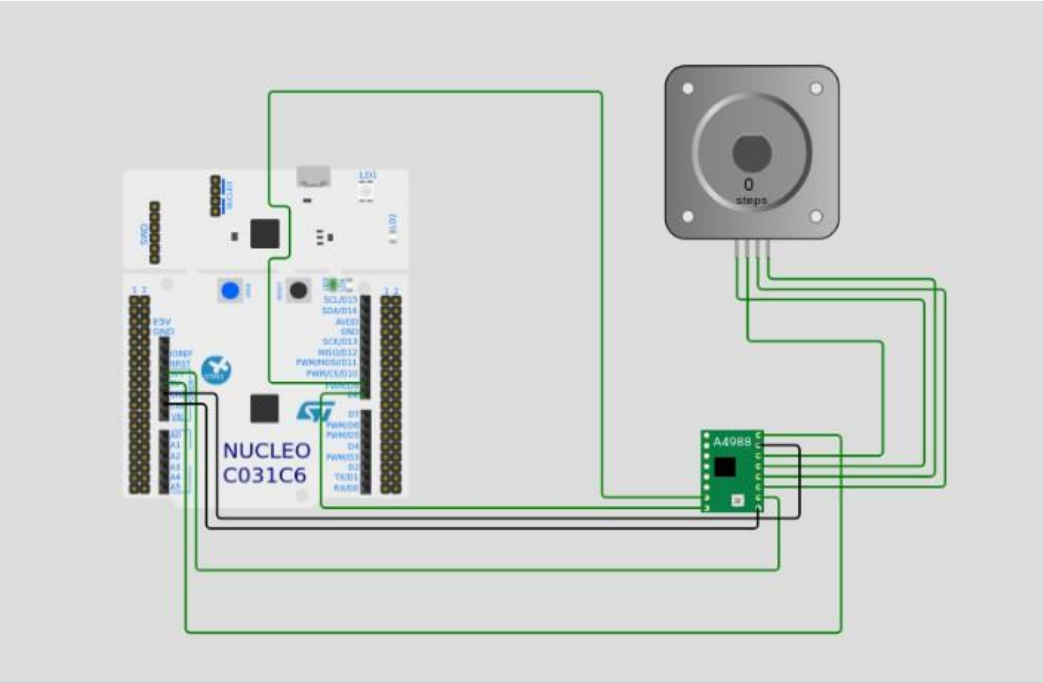
Step 1: Configure GPIO pins for motor control.
Step 2: Set signals for forward direction.
Step 3: Apply delay for motion.
Step 4: Change signals for reverse direction.
Step 5: Apply delay again.
Step 6: Repeat continuously.

Code:

```
#include   "part_c_common.h"

void  Motor_Set(int  forward);

void Motor_Set(int forward) {

   if        (forward)        {

Gpio_Set(GPIOA, 0U, 1);

Gpio_Set(GPIOA, 1U, 0);

   }             else             {

Gpio_Set(GPIOA, 0U, 0);

Gpio_Set(GPIOA, 1U, 1);

   }

}

int main() {

Clock_EnableGpioA();

   Delay_InitSystick_1ms();

Gpio_ConfigOutputPushPull(GPIOA, 0U);

Gpio_ConfigOutputPushPull(GPIOA, 1U);

while(1) {

Motor_Set(1);
```

```
Delay_Ms(1000U);
Motor_Set(0);
Delay_Ms(1000U);
    }
}
```

Output:




```
--- STM32 Stepper Motor Controller ---
Direction: CW, Speed: 1000 Hz _ 2300049078
Direction: CW, Speed: 200 Hz _ 2300049078
Direction: CW, Motor Stopped.
Direction: CCW, Speed: 1000 Hz _ 2300049078
Direction: CCW, Motor Stopped.
Direction: CW, Speed: 1000 Hz   2300049078
```

Inference:

I learned how direction control is achieved using digital signals.This helped me connect software logic with real-world mechanical movement.

Result:

The DC motor operated successfully in both forward and reverse directions.

Ex no: 9	4x4 Keypad Interfacing With LCD Display
Date:	

Aim:

To interface a 4×4 keypad with an ARM Cortex board and display the pressed key value on an LCD.

Algorithm:

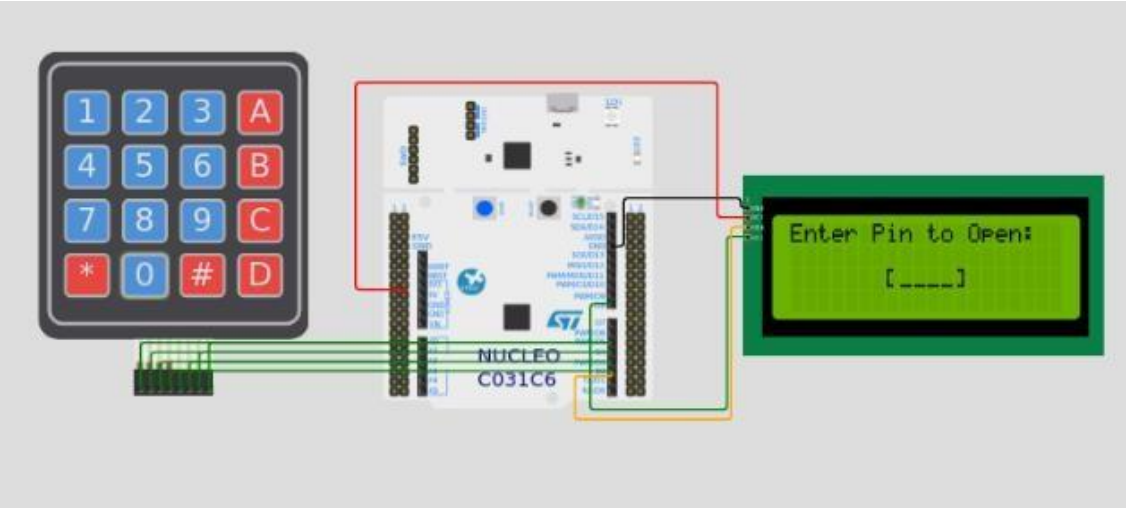
Step 1: Configure rows as output and columns as input.
Step 2: Activate one row at a time.
Step 3: Scan column inputs.
Step 4: Detect key press based on signal.
Step 5: Map the key value.
Step 6: Display or transmit the result.

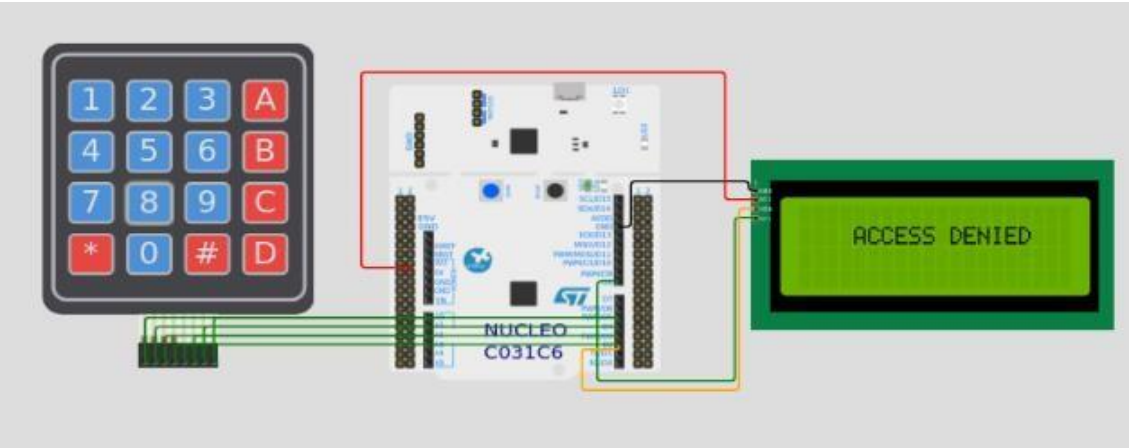
Code:

```
#include "part_c_common.h"

char Keypad_Scan(void);

char Keypad_Scan(void) {

    char map[4][4] = {

        {'1','2','3','A'},

        {'4','5','6','B'},

        {'7','8','9','C'},

        {'*','0','#','D'}

    };


    for (unsigned int r = 0U; r < 4U; r++) {

        for (unsigned int i = 0U; i < 4U; i++) Gpio_Set(GPIOA, i, 1);

Gpio_Set(GPIOA, r, 0);

Delay_Ms(1U);

        for (unsigned int c = 0U; c < 4U; c++) {

            if (Gpio_Get(GPIOA, 4U + c) == 0) return map[r][c];

        }

    }
```

```
    return 0;
}



int main() {
Clock_EnableGpioA();
    Delay_InitSystick_1ms();
    Usart1_Init_9600_8N1();

    for (unsigned int i = 0U; i < 4U; i++) Gpio_ConfigOutputPushPull(GPIOA, i);

    for (unsigned int i = 4U; i < 8U; i++) Gpio_ConfigInputPull(GPIOA, i, 1);
    Usart1_WriteString("Keypad scan (press keys)\r\n");

while(1) {
        char k = Keypad_Scan();
        if (k) {
            Usart1_WriteChar(k);
            Usart1_WriteString("\r\n");
            while (Keypad_Scan()) Delay_Ms(20U);
        }
Delay_Ms(50U);
    }
}
```

Output:





Inference:

I understood how matrix scanning is used to detect multiple inputs efficiently.This enhanced my knowledge of input device handling.

Result:

The pressed key was detected correctly and displayed on the LCD.

Ex no: 10	Relay Interfacing For External Lamp Control
Date:	

Aim:

To interface a relay with an ARM Cortex board to control an external lamp based on switch input.

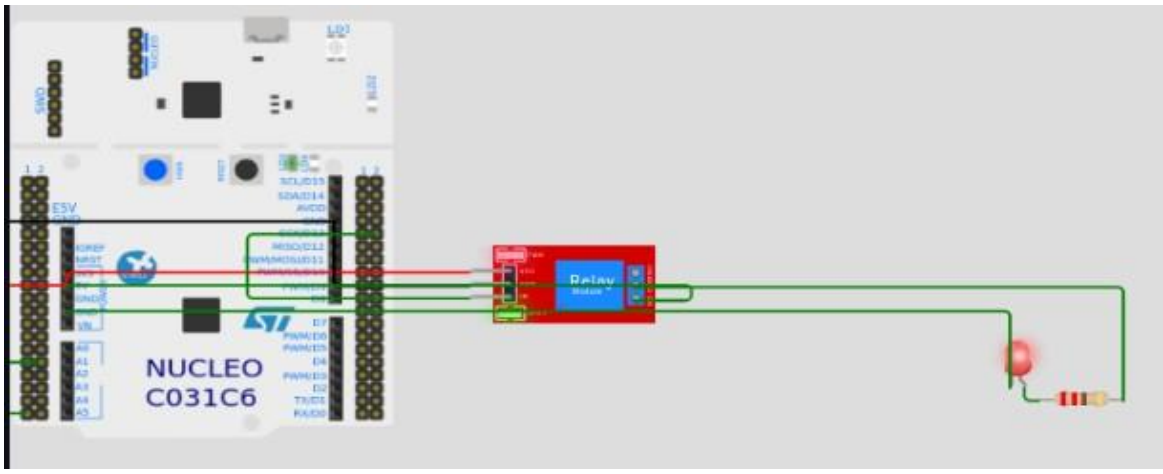
Algorithm:

Step 1: Configure relay control pin as output.
Step 2: Configure switch input pin.
Step 3: Read switch status.
Step 4: Toggle relay state on input.
Step 5: Control external device (lamp).
Step 6: Repeat continuously.

Code:

```
#include "part_c_common.h"

int main() {

Clock_EnableGpioC();

Clock_EnableGpioA();

   Delay_InitSystick_1ms();

Gpio_ConfigOutputPushPull(GPIOC, 13U);

Gpio_ConfigInputPull(GPIOA, 0U, 1);

   int relay_on = 0;

while(1) {

       int pressed = (Gpio_Get(GPIOA, 0U) == 0);

       if (pressed) {

relay_on= !relay_on;

Gpio_Set(GPIOC, 13U, relay_on ? 0 : 1);

           while (Gpio_Get(GPIOA, 0U) == 0) Delay_Ms(20U);

       }

Delay_Ms(20U);

   }

}
```

Output:



Inference:

I learned how embedded systems control high-power devices safely using relays.This improved my understanding of interfacing between low and high voltage systems.

Result:

The relay was operated successfully and the lamp was switched ON and OFF according to the input condition.

Ex no: 11	DHT11 Sensor Interfacing
Date:	

Aim:

To interface a DHT11 sensor with an ARM Cortex board and display temperature and humidity values on an LCD.

Algorithm:

Step 1: Initialize sensor communication pin.
Step 2: Send start signal to sensor.
Step 3: Switch pin to input mode.
Step 4: Read data bits from sensor.
Step 5: Verify checksum.
Step 6: Display temperature and humidity.

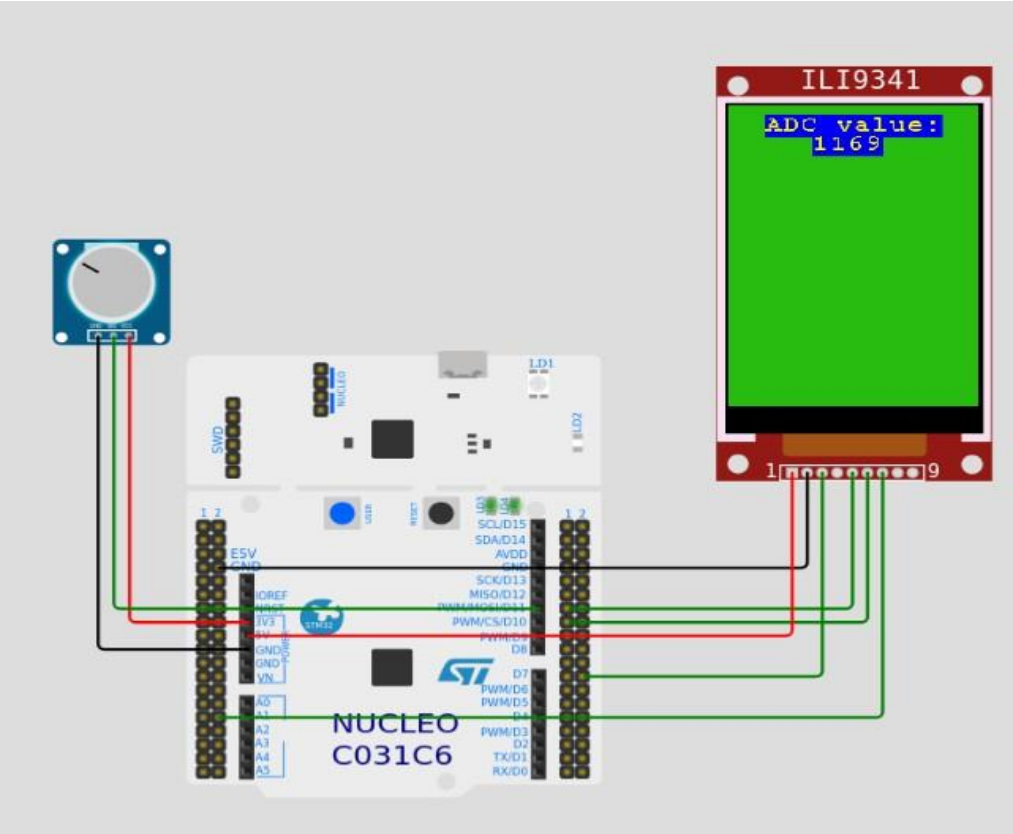
Code:

```
#include "part_c_common.h"

void Dht11_SetOutput(void);

void Dht11_SetInputPullup(void);

int  Dht11_ReadByte(unsigned int *out);

int  Dht11_Read(int *temp_c, int *humidity);

void Dht11_SetOutput(void) {

Gpio_ConfigOutputPushPull(GPIOA, 1U);

}

void Dht11_SetInputPullup(void) {

Gpio_ConfigInputPull(GPIOA, 1U, 1);

}


int Dht11_ReadByte(unsigned int *out) {

   unsigned int v = 0U;

   for (unsigned int i = 0U; i < 8U; i++) {

      unsigned int t = 0U;

      while (Gpio_Get(GPIOA, 1U) == 0) { if (++t > 80000U) return 0; }

      t = 0U;

      while (Gpio_Get(GPIOA, 1U) == 1) { if (++t > 80000U) break; }
```

```
        v <<= 1;
        if (t > 2000U) v |= 1U;
    }
    *out = v;
    return 1;
}

int Dht11_Read(int *temp_c, int *humidity) {
    unsigned int hI, hD, tI, tD, cs;
    Dht11_SetOutput();
Gpio_Set(GPIOA, 1U, 0);
Delay_Ms(20U);
Gpio_Set(GPIOA, 1U, 1);
    Dht11_SetInputPullup();
Delay_Ms(1U);
    unsigned int t = 0U;
    while (Gpio_Get(GPIOA, 1U) == 1) { if (++t > 80000U) return 0; }
    t = 0U;
    while (Gpio_Get(GPIOA, 1U) == 0) { if (++t > 80000U) return 0; }
    t = 0U;
    while (Gpio_Get(GPIOA, 1U) == 1) { if (++t > 80000U) return 0; }
    if (!Dht11_ReadByte(&hI)) return 0;
    if (!Dht11_ReadByte(&hD)) return 0;
    if (!Dht11_ReadByte(&tI)) return 0;
    if (!Dht11_ReadByte(&tD)) return 0;
    if (!Dht11_ReadByte(&cs)) return 0;
    if (((hI + hD + tI + tD) & 0xFFU) != (cs & 0xFFU)) return 0;
    *humidity = (int)hI;
    *temp_c = (int)tI;
```

```
    return 1;
}

int main() {
Clock_EnableGpioA();
    Delay_InitSystick_1ms();
    Usart1_Init_9600_8N1();
while(1) {
        int t, h;
        if (Dht11_Read(&t, &h)) {
            char buf[64];
            unsigned int n = (unsigned int)sprintf(buf, "DHT11: T=%dC H=%d%%\r\n", t, h);
            (void)n;
            Usart1_WriteString(buf);
        } else {
            Usart1_WriteString("DHT11 read failed\r\n");
        }
Delay_Ms(1200U);
    }
}
```

Output:



Inference:

I understood how sensors communicate using timing-based protocols.This helped me learn real-time data acquisition and validation.

Result:

Temperature and humidity values were measured successfully and displayed correctly on the LCD.

Ex no: 12	I2C LCD Interfacing
Date:	

Aim:

To interface an I2C LCD with an ARM Cortex board and display sensor values using I2C communication.

Algorithm:

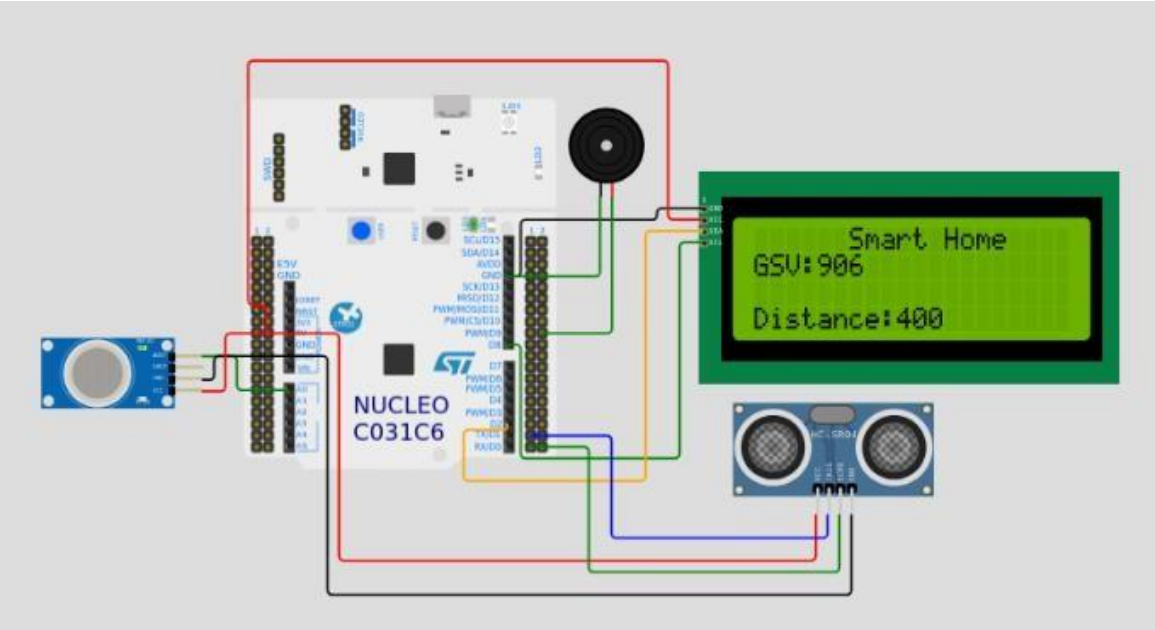
Step 1: Initialize I2C communication.
Step 2: Configure clock and control registers.
Step 3: Send start condition.
Step 4: Transmit device address.
Step 5: Send data bytes to LCD.
Step 6: Stop communication.

Code:

```
#include "part_c_common.h"

void I2c1_Init_100kHz(void);

void I2c1_WriteByte(unsigned char addr7, unsigned char data);

void I2c1_Init_100kHz(void) {

Clock_EnableGpioB();

   RCC->APB1ENR |= RCC_APB1ENR_I2C1EN;

   {

      unsigned int v = GPIOB->CRL;

      v &= ~(0xFU << (6U * 4U));

      v |= (0xFU << (6U * 4U));

      v &= ~(0xFU << (7U * 4U));

      v |= (0xFU << (7U * 4U));

      GPIOB->CRL = v;

   }

   I2C1->CR2 = 36U;

   I2C1->CCR = 180U;

   I2C1->TRISE = 37U;

   I2C1->CR1 = I2C_CR1_PE;
```

```
}
void I2c1_WriteByte(unsigned char addr7, unsigned char data) {
    I2C1->CR1 |= I2C_CR1_START;
    while (!(I2C1->SR1 & I2C_SR1_SB)) {}
    (void)I2C1->SR1;
    I2C1->DR = (unsigned int)(addr7 << 1);
    while (!(I2C1->SR1 & I2C_SR1_ADDR)) {}
    (void)I2C1->SR1;
    (void)I2C1->SR2;
    while (!(I2C1->SR1 & I2C_SR1_TXE)) {}
    I2C1->DR = data;
    while (!(I2C1->SR1 & I2C_SR1_BTF)) {}
    I2C1->CR1 |= I2C_CR1_STOP;
}
int main() {
    Delay_InitSystick_1ms();
    I2c1_Init_100kHz();
while(1) {
        I2c1_WriteByte(0x27U, 0x00U);
Delay_Ms(500U);
    }
}
```

Output:



Inference:

I learned how serial communication reduces wiring complexity.This gave me insight into efficient device communication techniques.

Result:

Sensor data was transmitted through I2C communication and displayed properly on the LCD.

Ex no: 13	PWM Generation Using Timer For Motor Speed Control
Date:	

Aim:

To generate a PWM signal using the Timer module of an ARM Cortex board to control the speed of a motor.

Algorithm:

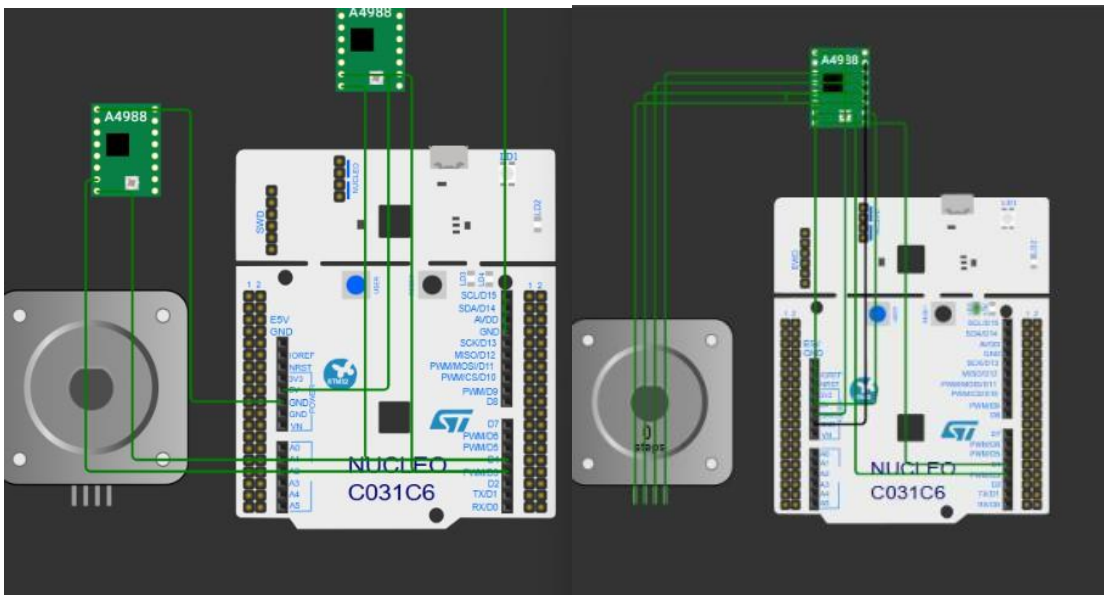
Step 1: Initialize timer module.
Step 2: Configure PWM mode.
Step 3: Set initial duty cycle.
Step 4: Generate PWM signal.
Step 5: Vary duty cycle gradually.
Step 6: Control motor speed accordingly.

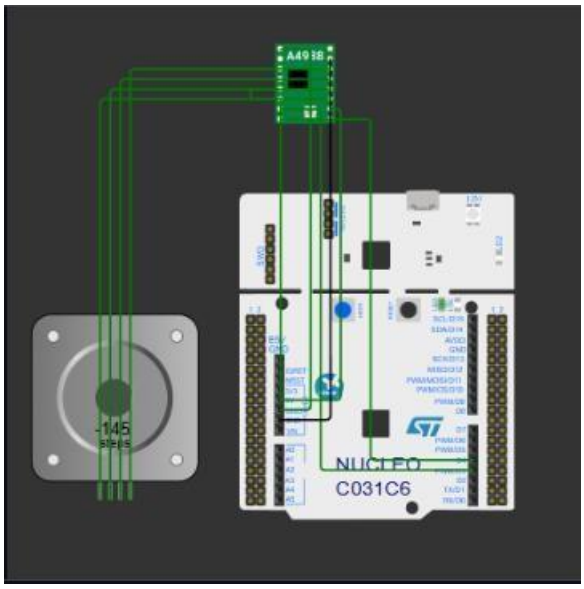
Code:

```
#include "part_c_common.h"

void Pwm_Tim2Ch1_Init(void);

void Pwm_Tim2Ch1_SetDutyPermille(unsigned int permille);

void Pwm_Tim2Ch1_Init(void) {

Clock_EnableGpioA();

   RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;

   {

      unsigned int v = GPIOA->CRL;

      v &= ~(0xFU << (0U * 4U));

      v |= (0xBU << (0U * 4U));

      GPIOA->CRL = v;

   }

   TIM2->PSC = 72U - 1U;

   TIM2->ARR = 1000U - 1U;

   TIM2->CCR1 = 0U;

   TIM2->CCMR1 = (6U << 4U) | TIM_CCMR1_OC1PE;

   TIM2->CCER = TIM_CCER_CC1E;

   TIM2->CR1 = TIM_CR1_ARPE | TIM_CR1_CEN;
```

```
}
void Pwm_Tim2Ch1_SetDutyPermille(unsigned int permille) {
    if (permille > 1000U) permille = 1000U;
    TIM2->CCR1 = permille;
}
int main() {
    Delay_InitSystick_1ms();
    Pwm_Tim2Ch1_Init();
while(1) {
        for (unsigned int d = 0U; d <= 1000U; d += 50U) {
            Pwm_Tim2Ch1_SetDutyPermille(d);
Delay_Ms(120U);
        }
        for (unsigned int d = 1000U; d > 0U; d -= 50U) {
            Pwm_Tim2Ch1_SetDutyPermille(d);
Delay_Ms(120U);
        }
    }
}
```

Output:





Inference:

I understood how speed control is achieved by varying duty cycle.This helped me relate signal modulation with physical output control.

Result:

PWM signal was generated successfully and the motor speed was controlled as required.

Ex no: 14	UART Communication Between Two ARM Cortex Boards
Date:	

Aim:

To implement UART communication to send and receive characters between two ARM Cortex boards.

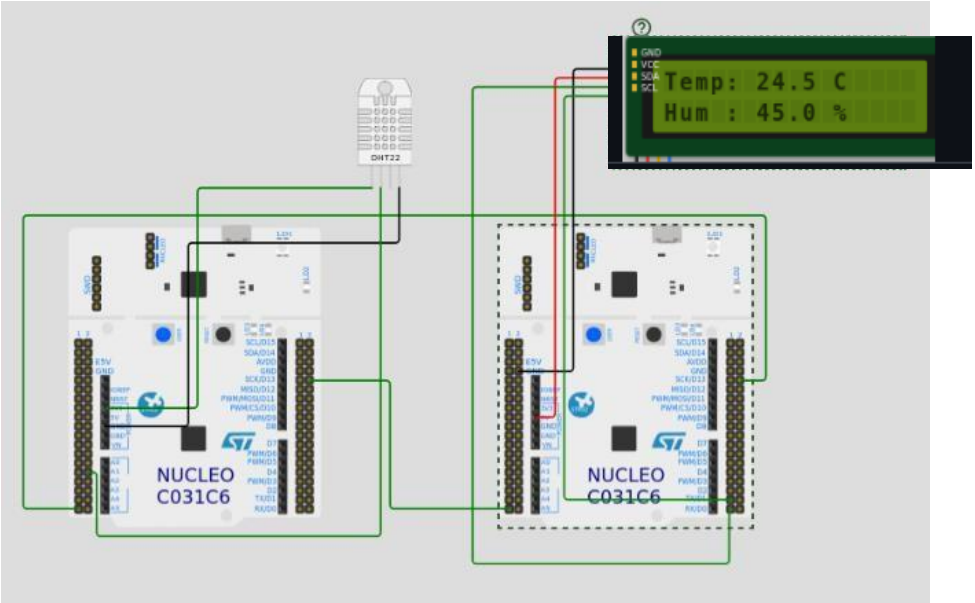
Algorithm:

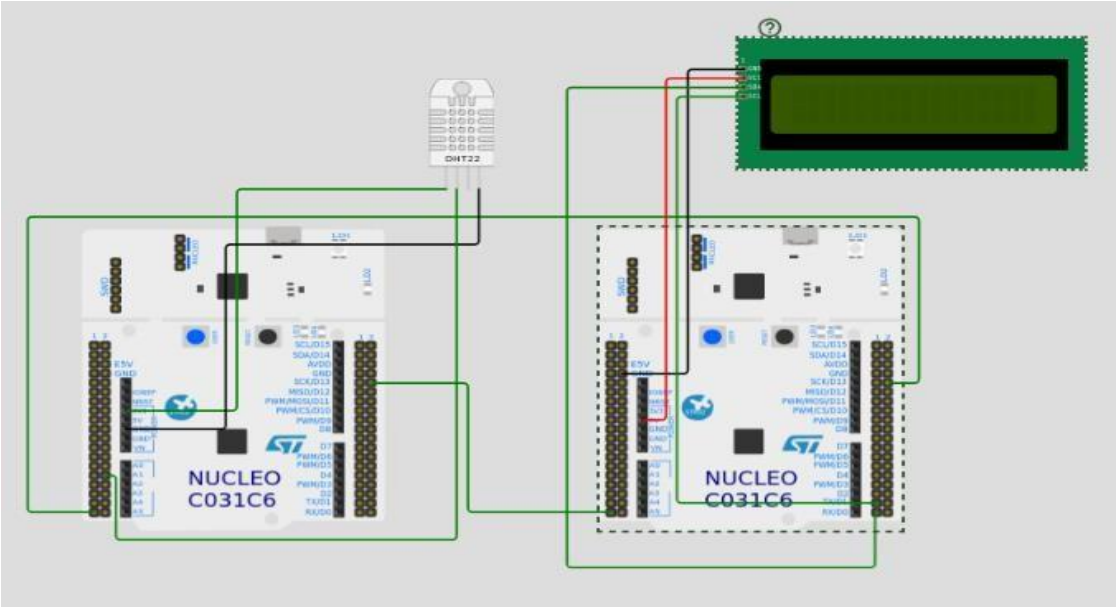
Step 1: Initialize UART module.
Step 2: Configure baud rate and settings.
Step 3: Send initial message.
Step 4: Read incoming data.
Step 5: Echo received data back.
Step 6: Repeat continuously.

Code:

```
#include "part_c_common.h"


int main() {

   Delay_InitSystick_1ms();

   Usart1_Init_9600_8N1();

   Usart1_WriteString("USART1 echo ready\r\n");

while(1) {

      char c = Usart1_ReadCharBlocking();

      Usart1_WriteChar(c);

   }

}
```

Output:





Inference:

I learned how serial communication enables data exchange between systems.This improved my understanding of communication protocols.

Result:

Serial data transmission and reception were carried out successfully between the boards.

Ex no: 15	ADC Interfacing For Analog Sensor Reading
Date:	

Aim:

To interface an analog sensor with an ARM Cortex board and read the sensor input using ADC.

Algorithm:

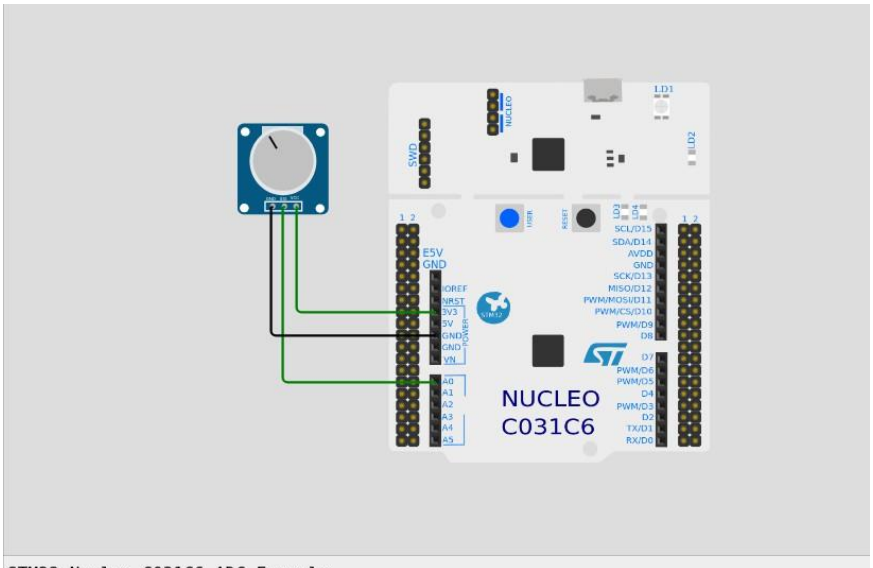
Step 1: Initialize ADC module.
Step 2: Select input channel.
Step 3: Start conversion.
Step 4: Wait for conversion completion.
Step 5: Read digital value.
Step 6: Display or transmit result.

Code:

```
#include "part_c_common.h"

void Adc1_Init_Channel0(void);

unsigned int Adc1_Read_Channel0(void);

void Adc1_Init_Channel0(void) {

Clock_EnableGpioA();

   RCC->APB2ENR |= RCC_APB2ENR_ADC1EN;

   GPIOA->CRL &= ~(0xFU << (0U * 4U));

   ADC1->CR2 = ADC_CR2_ADON;

Delay_Ms(1U);

   ADC1->SMPR2 |= (7U << 0U);

   ADC1->SQR3 = 0U;

}

unsigned int Adc1_Read_Channel0(void) {

   ADC1->CR2 |= ADC_CR2_ADON;

   ADC1->CR2 |= ADC_CR2_SWSTART;

   while (!(ADC1->SR & ADC_SR_EOC)) {}

   return ADC1->DR;

}
```

```
int main() {
    Delay_InitSystick_1ms();
    Usart1_Init_9600_8N1();
    Adc1_Init_Channel0();
while(1) {
        unsigned int v = Adc1_Read_Channel0();
        char buf[64];
        (void)sprintf(buf, "ADC0=%u\r\n", v);
        Usart1_WriteString(buf);
Delay_Ms(500U);
    }}
```

Output:




```
STM32 Nucleo C031C6 ADC Example
Rotate potentiometer to see ADC values.
ADC: 1045       Voltage: 0.842 V
ADC: 4095       Voltage: 3.300 V
ADC: 0  Voltage: 0.000 V
ADC: 0  Voltage: 0.000 V
ADC: 0  Voltage: 0.000 V
ADC: 636        Voltage: 0.513 V
ADC: 1457       Voltage: 1.174 V
ADC: 1577       Voltage: 1.271 V
ADC: 1577       Voltage: 1.271 V
ADC: 1577       Voltage: 1.271 V
```

Inference:

I understood how analog signals are converted into digital data.This helped me connect real-world signals with digital processing.

Result:

Analog input was successfully converted into digital data and the sensor value was obtained correctly.